CS301 Short Notes

Meta AI

M T W T F S

DATE: 11-4-25

CS301
Data Structures

Definition:-

Data structures help us to "organize the data" in the computer, resulting in more efficient programs. (help us to easily retrieve)

For example:

If things in a room are spreadout randomly, it's hard to find what you need. But everything is organized, it's easy to access.

Organizing the data:

It means that the data should be "arranged in a way" that is "easily accessible".

Selecting a data Structure: (how to solve a problem?)

First of all we have to "analyze the problem" to determin the resource

Second is how to solve & then write the steps for solve the problem.

ALgorithm

The steps that you write to solve a problem is called Algorithm.

An algorithm is a step-by step procedure to solve a problem or perform a task.

For Paid Service : +923174545120
CS301 Short Notes
By Qalb-e-Momin

Data Structure Philosophy:

First each data structure has costs (means Kitni cost lagy gi)

Second is benefits (means kya kya benefit mily ga data sturcture ko brany ma)

A data Structure requires: (3 main points)

— Space for each data item it stores

— Time to perform each basic operation

— Programming Effort

Goals of this course:

- Arrange the data in efficient / well-defined way. (organized the data)
- Save time (Reduce the time)
- Reduce the size of space (little space utilized)
- Increase (Your) the benefits.
  (Benefits ko high level pr ly kr jana)

Array:

An array is a collection of similar data types stored at contiguous memory locations.

- Using a single variable name for store and access multiple values.

For example:

```
int x [6] ;
```

data-type ← int
variable name ← x
size of array ← [6]
; ← statement terminator

How many cells allocate in memory?

Ans: 6 (size of array)

* Starting Index from zero.
* Maximum Index = size of array - 1 = 5
* Size of array is 6 here. [6]

List Programming:

The List data structure is among the most generic of data structures.

In daily life; shopping list, groceries list etc.

In this course, we will see how we use Lists in programming.

- It is possible to insert new elements at various position in the list & remove element of the list.
- In list programming, order of element is important/ crucial.

For example,

```
list = [5, 10, 15]
cout << list [0];        // output: 5
```

If you change the order output will also changed.

Lecture no 2:

List Implimentation

A list can be implemented using:

1- Arrays:

Fixed size, element stored in contiguous memory.

Provide fast access to elements using index.

2- Linked lists:

Dynamic size, elements (nodes) are connected using pointers.

Flexibility in size, Access is slower.

For example.

The list of integers (2,6,8,7,1) can be represented in the following manner where the current position is 3.

A	2	6	8	7	1		Current	Size
	1	2	3	4	5		3	5

For Paid Service : +923174545120
CS301 Short Notes
By Qalb-e-Momin

add Method:

adding the element (9) to the list ; the list will be (2, 6, 8, 9, 7, 1)

Procedure:

To add the new element (9) to the list at current position, at first, we have to make spare... we shift every element on the "right of" 8 ("current position"), after creating the space; put the element at empty space.

A	2	6	8	9	7	1			current	size
	1	2	3	4	5	6			4	6

i.e, position 4.

no. of element in the list

next Method:

Next method moves the current position one position forward. In this method, we don't add a new element to the list but simply move the pointer one element ahead.

(next)

remove Method:

The "remove" methode removes the element residing at the current position. (current position 5 that has value 7). In this method, when we remove an element, the element next to it on the right hand side comes as it place. And the remaining are also shifted one place to the right. Represented by the following. ←

A	2	6	8	9	1				current	Size
	1	2	3	4	5				5	5

find Method:

The find(x) function is used to find a specific element in the array. We pass the element, which is to be found, as an argument to the find function. After the traverses, "if the element is found", this function sets the current position to it & returns 1 i.e. true.

"if the element is not found", the function returns 0 i.e. false.

M T W T F S

7

DATE:

For Paid Service : +923174545120
CS301 Short Notes
By Qalb-e-Momin

```
#include <iostream>
#include <conio.h>
using namespace std;
main ()
{
    int a [5] = { 10, 20, 30, 40, 50};
    int found = 0, search-value;
    cout << "\n Enter value For Search: ";
    cin >> search-value;
    for (int i=0; i<=4; i++)
    {
        if (search-value == a: [i])
        { cout << "\n\n value is found";
            found++;
            break;
        }
    }
    if (found == 0)
    {
        cout << "\n\n value is not found";
    }
    getch ();
    return 0;
}
```

get() Method:

This method is used to get the element from the current position in the array.

Syntax:

```
return A [ current ];
```

current is pointing to (current position) in the list A. A value is passed to this method as an argument. It puts the value at the current position.

```
A [ current ] = x ;
```

Length ():

This method returns the size of the list

Syntax:

```
return size ;
```

back ():

This method decreases the value of variable current by 1.

- It moves the current position one element back ward.

Syntax:

```
current --; / current = current - 1 ;
```

∴ -- (This is a decreement operator in C++).

start() Method:

This method sets the current position to the first element of the list.

- We "use" the index 1 for the strating position.
- We "don't" use the index 0 for the starting position.

Syntax:

```
current = 1;
```

end() Method:

This method sets the current position to the Last element of the list. (size)

Syntax:

```
current = size;
```

★ There is no need of any loop or other programming structures to perform the task.

★ These methods (get(), length(), back(), start(), end(), update() etc) are one-step methods.

Difference b/w Array & List

Using linked Memory.

Array Linked	List Linked
In an array, the memory cells of the array are linked with each other.	In list linked, various cells of memory, are not located continuously.

Q No 1: In array linked, what happens if we again need to store more elements?

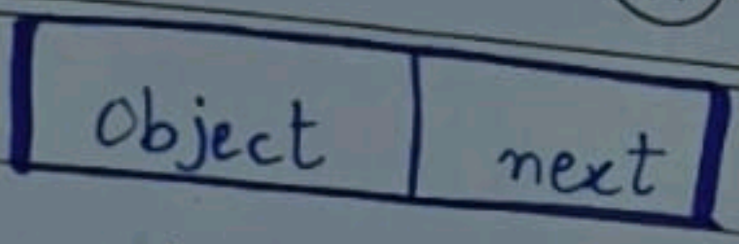
We will change the code of our program to change the declaration of the array while recompiling it.

In a linked list, if we need to store more elements, we can dynamically create a new node and link it to the list. There is no fixed size, so adding more element is easy.

Linked List

At first we define a node. A "node" comprises two fields. i.e. the object field that holds the actual list element. & the next holds the starting location of the next node.

DATE:



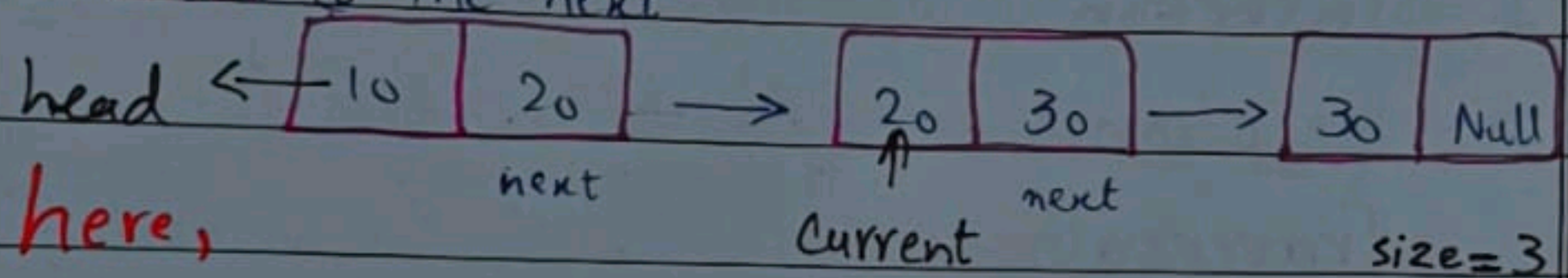
For example:

Suppose you have 3 values:

10 → 20 → 30

Each value stored in a node, and every node contains:

- Data (e.g 10)
- Pointer to the next



here,

each node points to the next one.

The last node points to NULL, which means the list ends here.

Linked List in memory

There are two parts; one the left and other is right.

On the left is the linked list chain that is actually the conceptual view of the linked list.

On the right is the linked list inside the computer memory with memory address.

Lecture No 3:

Linked List Operations

The linked list data structure provides operations to work on the nodes inside the list.

Basic operations performed on a linked list:

1 - Insertion — Add a new node.
2 - Deletion — Remove a node from any position.
3 - Traversal — Visit each node
4 - Search — Find a node with specific value.
5 - Update — Modify the data of a node.

1- add() operation:

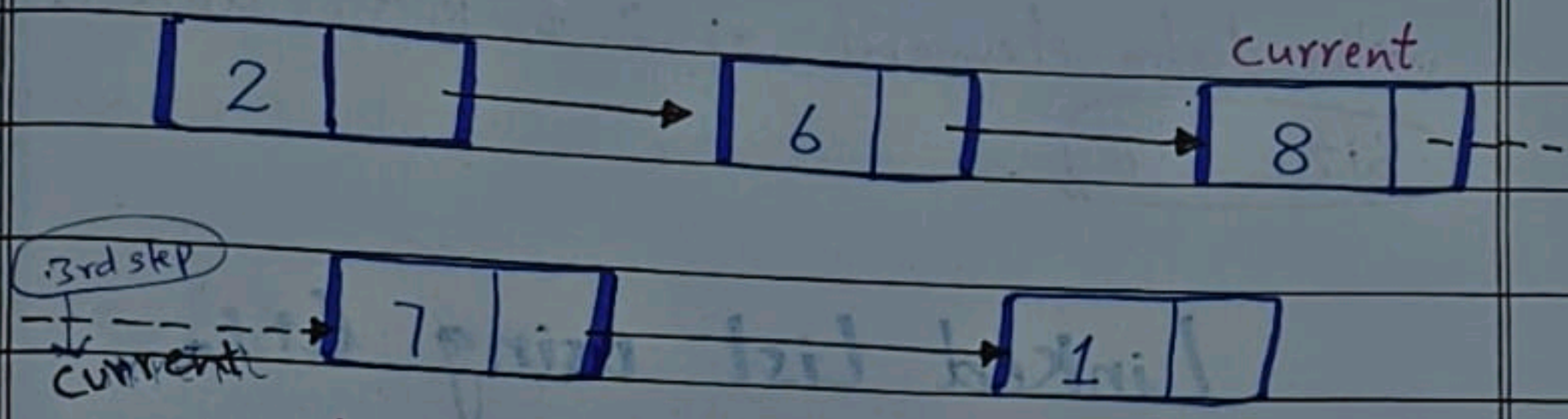
The Add (9) is used to create a new node in the memory at the current position to hold '9'.

Code in C++;

```
Node * newNode = new Node(9);
```

→ On the left,
Node is the data type, newNode is the variable pointer.
→ On the right; new operator is used to create newNode object as newNode(9).

Insertion of new node into the linked list:



Insert a new Node with data element 9. This new node will be inserted at the current position. The current pointer on the node with data element 8.

⊥ How we can insert a new node?

As we know every node has the address of the next node like; 2 has the address of 6 and 6 has address of 8 element node.

So, we changes the address.

1-Step:

To point next pointer of the new node (9) to the node (7).

2-Step:

To point next pointer of the node (8) to the new node (9).

3-Step:

To change the current pointer

to point to the new node (9).
Now, the updated list has nodes with data element as 2, 6, 8, 9, 7 and 1.
Size = 6.

Linked List using C++.

1- The Node class:

It begins with the word class.
In the body, we write its public and private data member and variables.

Two types of variable.

First one is object of type int, that store the data Part of the node.
Second is next Node, that store the address of next node.

-: Data member mention as Private.

∴ Singly-linked lists have one NULL Pointer.

The next field in the last node is set to NULL.

Public Functions:

get ():

This method returns back the value of the variable object.

set ():

Accepts a parameter of type int while returning back nothing.

get Next ():

It returns a pointer to an object of type Node lying somewhere in the memory. It returns next Node.

set Next ():

This method is used to connect the next node of the linked list with the current object. It is passed an address of the nex node.

For Paid Service : +923174545120
CS301 Short Notes
By Qalb-e-Momin

M T W T F S : : (16) DATE: _____

Lecture no 4

Methods of linked list:

1- Start Method:

This method is used for Position current Node and last current Node at first element.

```
Void start()
{
    LastCurrentNode = headNode;
    currentNode = headNode;
};
```

These two pointers point at different nodes of the list. Here we have pointed both of these pointer at the start of list.

- These pointers will move forward.
- These pointers cannot go behind .headNode.

* In the singly-linked list, we pointers can move only in one direction-

One Pointer

* Moving forward is easy is single link list but moving backward is difficult.

DATE:

2- **Remove Method:**

This method is used for remove the node from the linked list.

There are two scenarios:

- No.1 is delete or remove the first node.
- And second is delete/remove the any node from the linked list.

If you want to remove first Node from the linked list:

We use the remove method

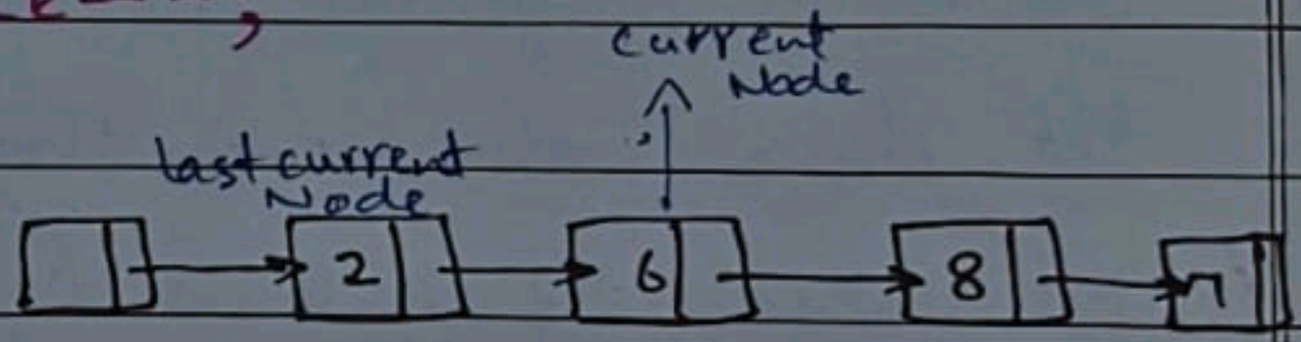
void remove () then apply condition if (head == NULL) show the msg (linked list is empty) on the screen. else get data (value) from the user for delete. and check the node is first or not and the next step is: if the node is first give the address of the next node to the ~~temt~~ ~~current node or~~ head

```
if (n == head -> getData())
  {
    Node * currentNode = head;
    head = head -> getNext();
    delete currentNode;
  };
main()
{ Obj. remove()
}
```

If you want to remove any Node from the linked list.

we use the method remove for this purpose.

```
Void remove () {
if (currentNode != NULL &&
        currentNode != headNode)
        {
(Step-1)    lastcurrentNod -> setNext (
            currentNode -> getNext());
(Step-2)    delete currentNode;
(Step-3)    currentNode = lastcurrentNode;
(" -4)      size--;
        }
};
```



Step-1 :

We will set the next pointer of the node pointed by the last current node with value 8.

∴ current node sy next node a address hm last current node ko dy dain gy.

Step-2.

Now, current nod is deleted.

Step-3.

Then, last current node into become a current node

Step-4.

When 1 node is deleted, size will be reduced.

```
size--; / size = size-1;
```
MCOS Both are Same

3 **Length Method.**

Length() returns the size of the list.

```
int length()
{
    return size; }
```

Analysis of Linked List:

We will be going to analyze each data structure. • It is useful or not.

• Its cost and benefit with respect to time and memory.

In the link list we check this during perform some task/methods.

- add
- find
- remove
- back.

M T W T F S

20

DATE:

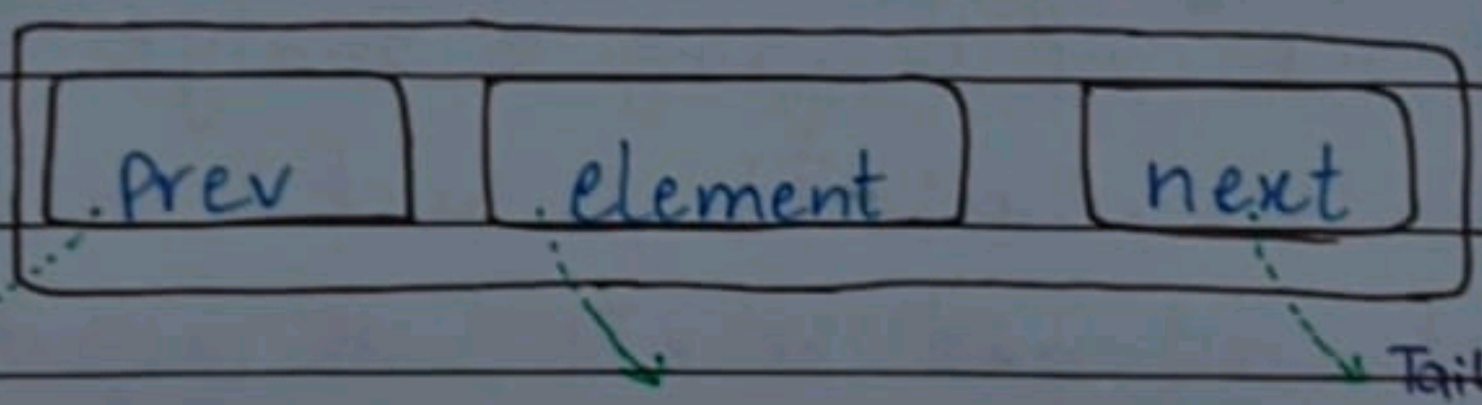
Doubly-Linked List:

Q: Do you need a list in which one has to move back or forward or at the start or in the end very often?

Answer:

If Yes, so we have to use double link list.

In doubly-link list, a programmer uses "two Pointers" in the node, i.e. one is next (next Point to the next node) and other is prev (Point to previous node). And "three parts".



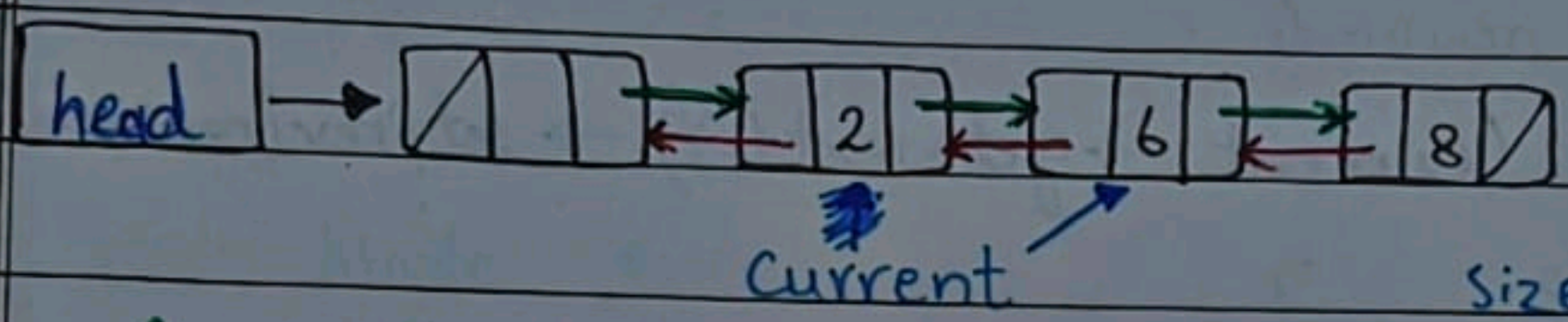
* We can move forward or backward in doubly-link list very easily.

↘ Have Two NULL Pointers.

ie. prev in the first node and next in the last node.

While adding or removing a node in a doubly linked list, The order in which pointers are recognized is important.

Pictorial View of doubly-link list:



Arrows pointing towards "right side" are representing "next node".
"Left side" are representing "prev nod".

Suppose we have created a new node:

Create a new object using new keyword.

1-step:

We assign the address of the node with value 8 to the next node of the new node.

```
new node → setNext( current → getNext() );
```

2-step:

In this, we assign the address of current Node to the prevNode of the newNode.

newNode → setprev(current);

3- step:

In this, we get the address of next Node to the current node and give the prevNode to the newNode.

(current → getNext()) → setprev(newNode);

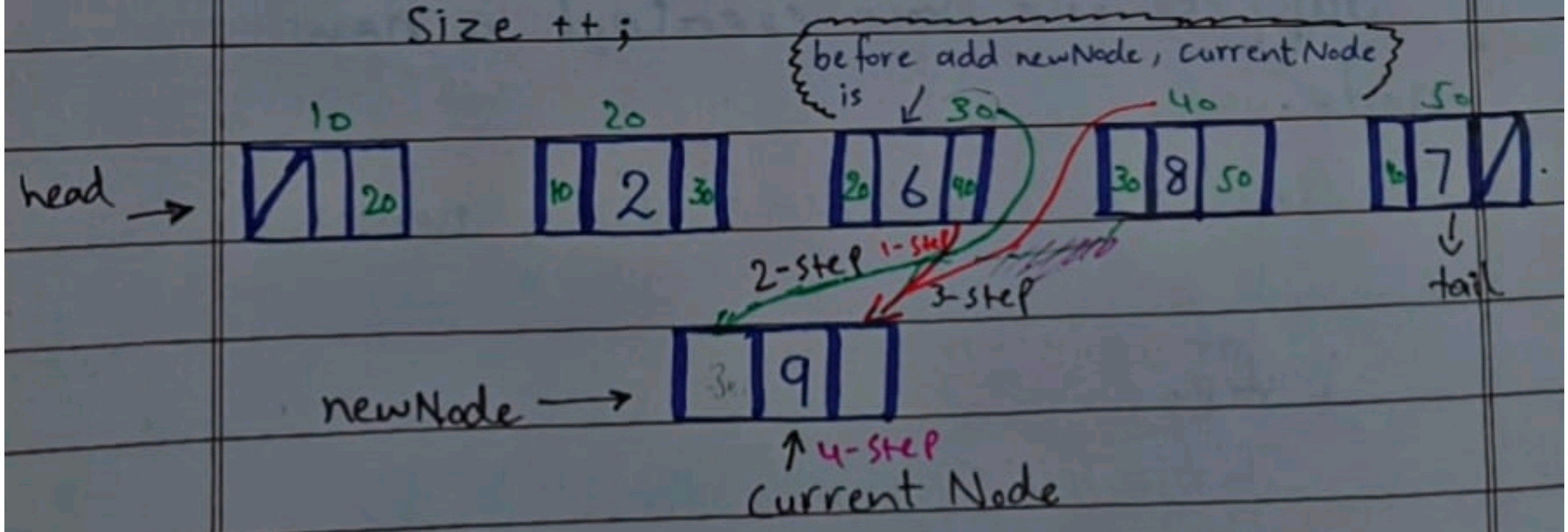
4- Step:

Point the current to the newNode and add one to the size of the list.

current → setNext(newNode);

current = newNode;

Size ++;



Now, the new node has been inserted between node with value 6 and node with value 8.

M T W T F S

(23) DATE:

Code for creating a New Node:

```
# include <iostream>
# include < conio.h >
Using namespace std;
Class Node
{
    Public:
        int data;
        Node   *prev;
        Node   *next;
        Void  SetData (int d)
        {
            data = d;
        }
        int  get Data () { return data; }
        Void  set prev (Node *p)
        { Prev = p; }
        Node *get prev () { return prev; }
        Void  Setnext (Node *n)
        { next = n; }
        Node * get next () { return next; }
```

M T W T F S

24

DATE:

For Paid Service : +923174545120

CS301 Short Notes

By Qalb-e-Momin

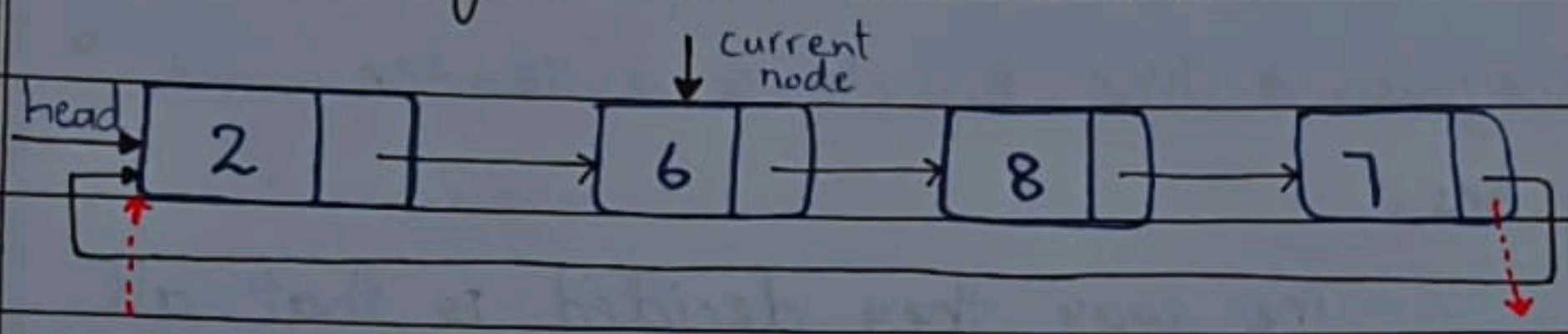
Blank

M T W T F S (25) DATE:

Circularly-linked lists

A way around this potential hazard is to "link the last node with the first node" in the list to create a circularly-linked list.

It means that the next of the last node is pointing towards the first node.



Point to the first node

Next of the Last node

Q: What is the benefit of this?

If you use the next or back methods (in doubly-linked list) that move the current Pointer, it will never point to NULL.

* Make the algorithm much faster & efficient.

Key Points:

- In a singly link list we move in one direction. Moving forward is "easy" but going back is "difficult" task.
- In a doubly link list we can move forward and backward very easily.

Problem:

the prev Pointer of the "first node" and next Pointer of the "last node" are NULL.

- In a circularly linked list there is no NULL.

Josephus Problem:

The problem is:

Consider there are 10 persons. They would like to choose a leader.

Ans:

The way they decided is that all lo sit in a circle. They start a count with Person 1 and go in clockwise direction and skip 3. Person 4 reached is eliminated. The count starts with the fifth and the next Person to go is the fourth in count.

Short Question // 2025

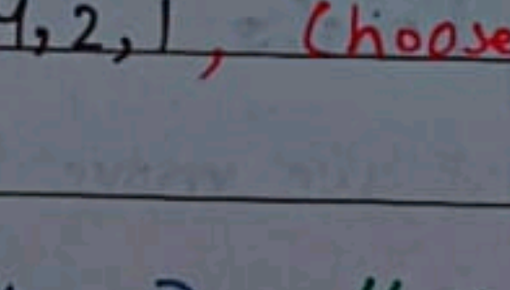
Q: Suppose you have the list of numbers 3,8,7,6,5,4,2,1, Choose the Leader.

Solution:

let, M = 3 // show the count

N = 8 // T.number of list

we have,



∴ Start counting, Skip 3, 4 eliminated

4 is the leader.

M T W T F S

(27)

DATE:

Lecture No 5:

Benefit of using circular linked list:

- In circular linked list, there is no NULL.
- It's used for resolution of the Josephus problem, can also be employeed in other problems.
- We can move easily and come to first node from anywhere.

CList: CList data structure

File ↙ of class CList and its object i.e. list.

The class CList of this circular linked list can be created (through which any number of objects of datatype of circular link list).

The class CList as a factory, creating as many objects of list as needed.

Methods: add, remove, next, back, get etc.

- It simplifies the code of the program.
- It can make the algorithm much faster and efficient.

```
# include "CList.cpp"
```

For Paid Service : +923174545120
CS301 Short Notes
By Qalb-e-Momin

M T W T F S ..

(28)

DATE:

Abstract Data Types

The term ADT refers to the basic mathematical concept that defines the datatype.

"A Programmer can change the internal implementation of the list and Program (using this list) will not be affected at all". This is the Abstract Data type (ADT).

→ These ADTypes are implemented using classes in C++.

→ The implementation of these ADTypes can be changed any time.

→ The C++ classes provide a programmer an ability to create such ADTs.

- Point to be remembered:

You should not worry about the internal implementation of these ADTs.

If we want to change the internal implementation of ADTs, it can be done without affecting the users of these ADTs.

While, writting a program, you should check it performance, if you feel that is slow, check the ADTs used at that point.

Stacks

Stack is a collection of elements arranged in a linear order. e.g. stacks of books, stack of plates, etc.

∴ We can add new items at the top of the stack or remove them from the top.

we can only access the element of the stack at the top.

In case of stack:

- We will not move forward or backward in the stack.
- We cannot suggest the removal of the middle element of the stack.

Stack Methods / Operations:

The methods of stack are very simple. There is no complexity involved. We do not need to carry a lot of work.

During the usage of array, the stack methods Push, Pop, top, isFull, isEmpty all are constant time operations.

5 methods

1- Push (x)

The Push(x) method will take an element and insert it at the top of the stack.

```
Void Push(int x)
{
   A[++current]= x;
}
```

2- Pop()

The Pop() method will remove the top element of the stack and return it.

void pop() ✗

int pop() ✓

{ return A [current--];}

3- Top()

The top() method returns the top element without removing it from stacks.

int (top) ✗

int top() ✓

{ return A [current];}

4- isFull()

The isFull() method return the boolean value. If this method return true, it means that the stack is Full & no more element "can" be inserted.

Before calling the Push(x), the user should call is Full() method.

If isFull() Returns False, it means: Stack is not Full and element can be inserted.

```
int isFull ()
{
  return (current == size -1);
}
```

5- isEmpty()

The isEmpty() method return true if the stack is empty and false otherwise.

it means there is no element in the stack.

Before calling the pop(), user should call isEmpty() method.

If isEmpty() returns False, call the pop(), otherwise, there will be a problem.

```
int isEmpty()
{ return (current == -1); }
```

Remember:

The last element to go to into the stack is first to come out.

A stack is known as LIFO.

Last In First out

At the end, جب ہم کوئی add value کروائیں گے تو وہ شروع میں آجائے گی

Top value

M T W T F S

32

DATE:

Stack Implementation using array

Stack:

A stack is a Linear data structure that follows the LIFO (Last In, First Out) Principle.

When implemented using an array, we do the following:

- Use an array to store element.
- Maintain an integer variable top to know where the last (top) item is.
- If top = -1, the stack is empty.
- On push, we increase top and add value.
- On Pop, we remove the value at top and decrease it.
- Arrays are of fixed size. And when they become full no more elements can be added to them.

→ is Full() method is used there, bcz of Limitation of array.

Stack Using Array

67232 65536

Lecture No 6:

Stack Using Linked List

A stack carries fifo behavior i.e. Last in, first out.

Why we use linked List instead of Array?

We can avoide the size Limitation of a stack implemented with an array, with the help of a linked List to hdd the stack element.

Because in stack implementaition with an array having fixed size. To avoid this, we use Linked List instead of array.

Stack Implementation

If we use linked list to implement the stack, few facts to consider:

- For a singly linked list, insert at start or end takes Constant times.
- Removing an element at start is Constant time but at end requires traversing the list to the 2nd last node. So, removing from the start is better approach rather than from the end.

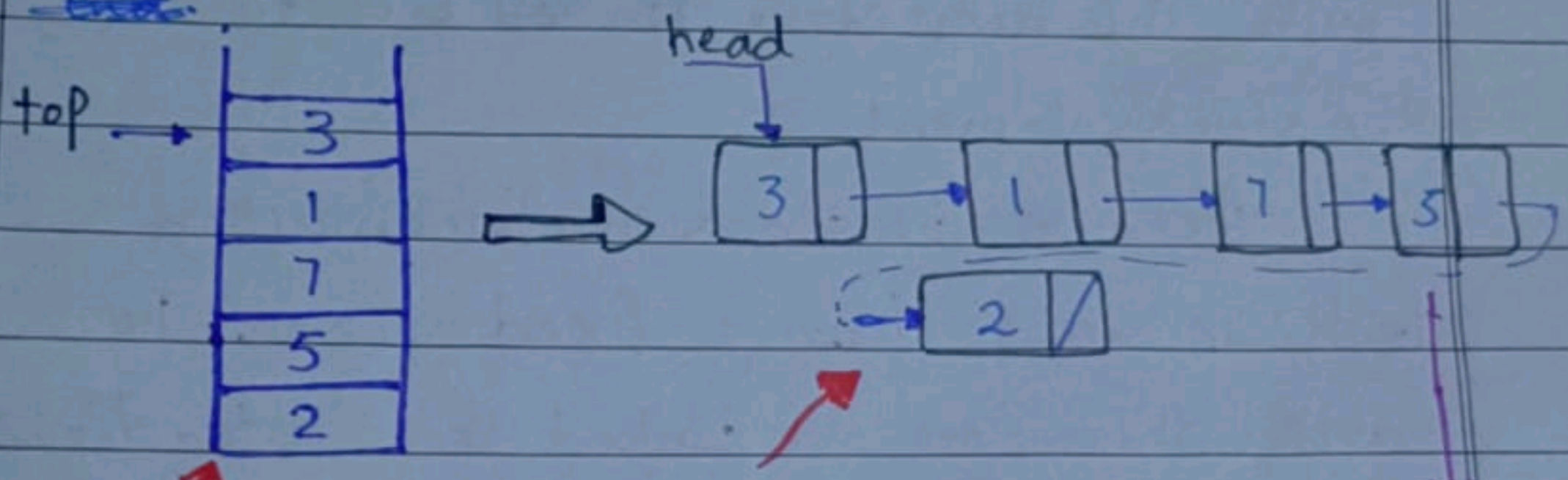
Imp:

Stack using linked list - there is no requirement of doubly or circularly linked list.

↙

In, we can move forward & backward easily Singly linked list can serve the purpose.

Hence, we use Push & Pop for insert & remove element from the list at start.



Stack using array

Stack implementation Using linked list.

↳ The most recent element of the stack is [3].

On the right hand side, there is a stack implemented using a linked list. This stacks has 4 nodes.

For Paid Service : +923174545120
CS301 Short Notes
By Qalb-e-Momin

Stack Methods / Operations

There are two Primary methods of a stack:

- Push() // For inserting element
- Pop() // For removing element

There are four operations:

- Push()
- Pop()
- top()
- isEmpty()

All are take constant time. These are very simple methods & don't contain loops.

Note:

We have not written "isFull()" while implementing stack with the linked list.

Use of stack

Uses of stack including - traverse and evaluate prefix, infix, Postfix expressions.

infix expressions.

The operater is written / present between two operands called infix expressions.

we can write as: A + B

Prefix expressions:

The operator is written before the operands A and B. These kinds of expressions are called Prefix expressions.

we can write as: +AB

Postfix expressions:

The operator is written after the operands A and B. This expression is called Postfix expression.

AB+

Precedence of Operators:

There are 5 binary operators, called addition, subtraction, multiplication, division and ~~exfot~~ exponentiation.

∴ All relational operators are binary once.

The rules or order of execution of operators in Mathematics called Precedence.

The order of Precedence is (highest to lowest):

M T W T F S

Lecture No 7

DATE:

Exponentiation ↑

Multiplication / division *, /

Addition / Subtraction +, -

- For operator of same precedence, the left to ~~wo~~right rule applies:
  (A+B+C) means (A+B)+C

- For exponentiation, the right-to-left rule applies:
  A ↑ B ↑ C means A ↑ (B ↑ C)

Convert the Infix to Postfix:

Infix	Postfix
A + B	A B +
12 + 60 - 23	12 60 + 23 -
(A+B) * (C-D)	AB + CD - *
A ↑ B * C - D + E / F	AB ↑ C * D - E F / +
4 + 3 * 5	4 + 35 * → 435 * +
(4+3) * 5	43 + 5 *

Solve کو '*' ہے۔ According to Precedence اس میں expression

highest to lowest ۔ کو '+' پھر کریں گے۔

If we want this, first "+" operator evaluate

we have to use the parenthesis ().

Where we use the Parenthesis and why?

We use the parenthesis () in the infix expression but we are not Using this () in Postfix expression.

Reason:

If we have to evaluate the Lower precedence operator before the higher Precedence operator.

For example:

(A+B)*C

∴ This means that we have to evaluate + before the multiplication.

The objective of using Parenthesis is to establish precedence.

Infix to Postfix Conversion: Using Stack:

* / High Priority

\+ - Low Priority

Rules:

1- Same Priority

2- Before high priority & After low Priority

3- Close Brackets

Expression:

(A + B / C * (D + C) - F)

'infix expression

To convert, we follow the rules.

Input/symbol	Stack	Postfix
(	(	
A		A
+	(+	
B	(+	AB
/	(+ /	
C	(+ /	ABC
*	(+ / *	ABC/
(	(+ / * (	ABC/
D	(+ / * (	ABC/D
+	(+ / * (+	ABC/D
C	(+ / * (+	ABC/DC
)	(+ / *	ABC/DC+
−	(−	ABC/DC+*+
F	(−	ABC/DC+*+F
)		ABC/DC+*+F−

When closing bracket come move right to left, pick all operater, until you find open bracket (.

Infix to Postfix expression is:

ABC/DC+*+F−

ABC/DC+*F−

Expression:

4+3*5

Input	Stack	Postfix
4		4
+	+	4
3		43
*	+*	43
		435*+

now, we have to check rule, which rule APPlied: here no rule applied.

5

infix to Postfix expression is: 435*+

Expression:

A+B*C

Input	Stack	Postfix
A		A
+	+	A
B	+	AB
*	+*	AB
C	+*	ABC
		ABC*+

infix to Postfix expression is: ABC*+

Lecture No 8:

C++ Templates:-

In C++ Language, a template is a function or class that is written with generic data type. `template <classT>`

When a programmer uses this function or class, the generic data type is replaced with the data type.

A template can be understood with the example of factory that bakes biscuits.

The factory may use flour, corn or starch as ingredients of the product. But the process of backing biscuits is the sam, whatever ingredients it uses.

factory اگر آپ مختلف ingredients استعمال کریں پھر بھی

biscuits بنے گا۔ same Process میں

So, we call the factory as the template for the biscuits.

Q. Is there any method to write the code for the stack once & then use it for different types of data?

Ans: Yes, For different stacks of different data types, we write once code as a template and reuse it for creating different stacks.

M T W T F S DATE:

template <class T>

Here T is a variable name for generic data type. A data type will replace T whenever template is used in the program.

int Push (T &);

int is the datatype in the Push function. Now, there is T written as the datatype in the Push () function.

Here, T is a generic data type

* The templates are so important that C++ Provides a library in which a large number of functions are provided as template. This library is a part of the official standard of C++. It is called STL.

STL: Standard Template Library.

Function Call Stack:

It is another example of stack. Whenever a programmer calls a function, he or she passes some arguments to parameters to the function. The function does work on these arguments and returns a value to the calling function or Program. This is known as return value

of a function.

For Paid Service : +923174545120

CS301 Short Notes

By Qalb-e-Momin

M T W T F S

DATE:

Lecture No 9:

Queues:

A queue is a linear data structure. In which items can only be inserted at one end (جہاں سے ہمیں value add کروانی ہے تو وہاں سے ہی ہم add کریں گے۔) and removed from the other.

Stack is a LIFO structure.

Queue is a FIFO structure.

(or)

First-In-First-out.

Queue Operations:

The queue data structure supports the following operations:

1- enqueue(X):

Used for inserting tha value into queue. Place X at the rear (last) of the queue.

2- dequeue():

Used for removing the front (top) element and return it.

3- front():

Return front (top) element without removing it.

M T W T F S

DATE:

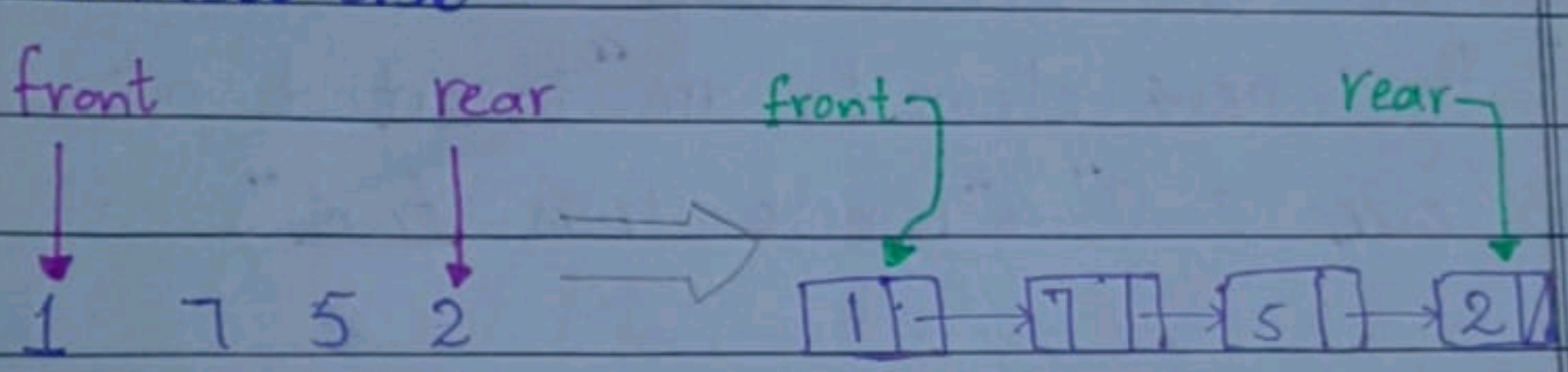
4- isEmpty():

Return TRUE if queue is empty, FALSE otherwise.

Implementing Queue:

we have two methods for implementing queue: with the help of Linked-list and with the help of array.

Queue Implementation using linked list:



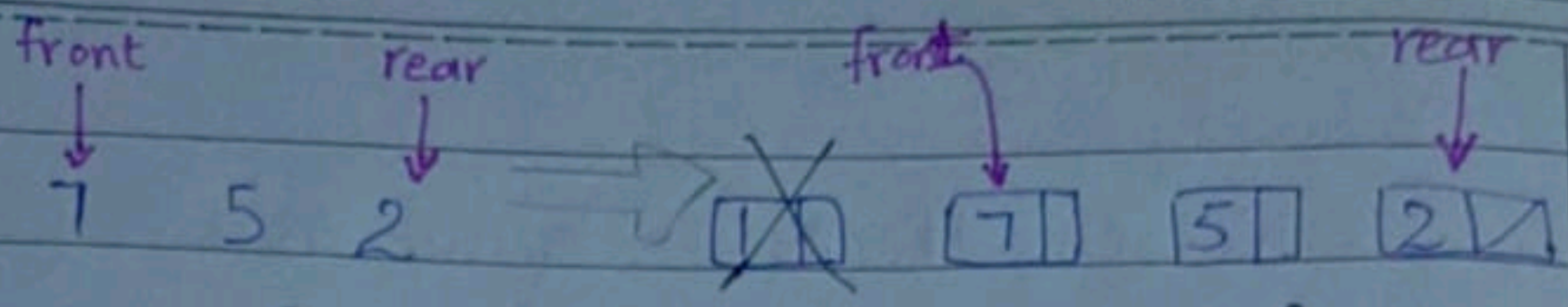
queue elements on the left side with two pointers front and rear.

first element ← → Last element

After dequeue() is called once:

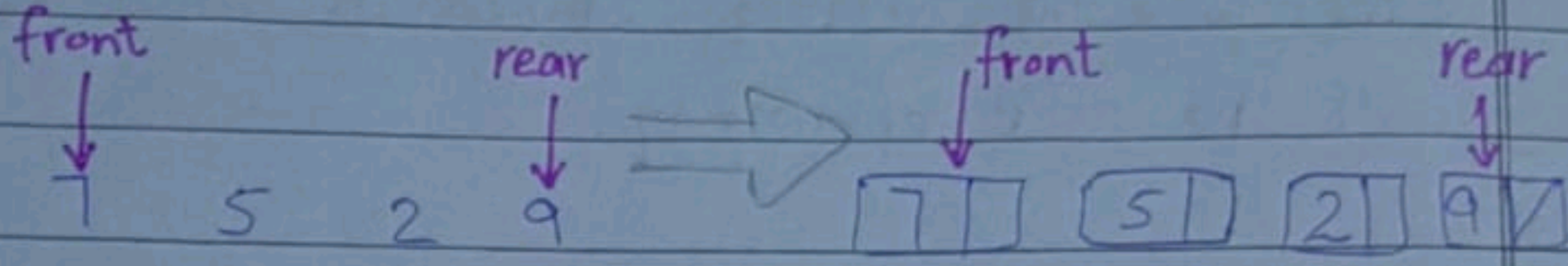
The front element 1 is removed from the queue. Note that front pointer moved to next element 7 in the list after removing the front element 1.

M T W T F S DATE:



Removal of one element from queue using dequeue().

→ After enqueue (9) call.



In this queue data structure, the new elements are "inserted at rear end" & "remove from front".

جب بھی آپ کوئی value کی مدد سے enqueue() insert کرواتے ہیں تو وہ queue میں insert پہ end ہوگی۔ اور سب سے end / last والی value کو ہم rear کہتے ہیں۔

```
void enqueue (int x)
{
    Node * newNode = new newNode;
    newNode → set(x);
    newNode → setNext(NULL);
    rear → setNext(newNode);
    rear = newNode;
}
```

~~After D.~~

dequeue () : Remove element from the front and return it.

جب بھی آپ کوئی queue value میں سے remove کرتا چاہتے ہیں اُس کے لیے dequeue() کا استعمال کرتے ہیں۔ یہ، یہ پہلی value جو queue میں پڑی ہوئی ہے اُسے remove کرتا ہے جسے ہم یہاں front کہتے ہیں۔

```
int dequeue ()
{
    int x = front -> get();
    Node *p = front;
    front = front -> getNext();
    delete p;
    return x;        // return it
}
```

→ front () method: Used to retrive the front element.

queue میں سب سے پہلے element کو get کرنے کے لیے front() استعمال ہوتا ہے۔

```
int front ()
{
    return front -> get();
}
```

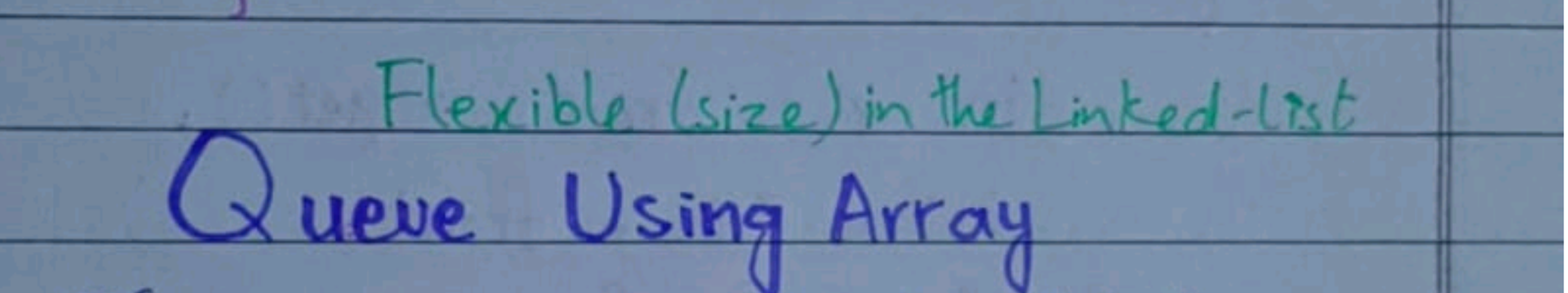
isEmpty(): This method is used to check whether the queue is empty or not. It checks the address inside the front pointer, if it is NULL, It will return TRUE, it means queue is empty.

```
int isEmpty()
{
    return (front == NULL);
}
```

Flexible (size) in the Linked-List

Queue Using Array

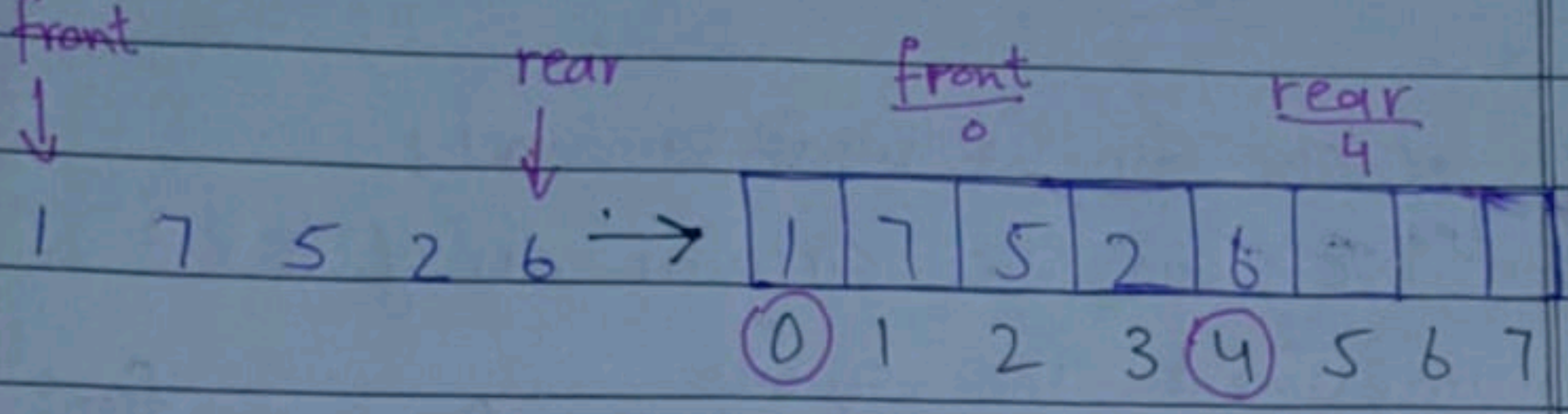
- If we use an array to hold the queue elements, both insertion and removal at the front (start) of the array are expensive.



Queue implementation using array.

Here, the size of array is 8. index from zero to 7.

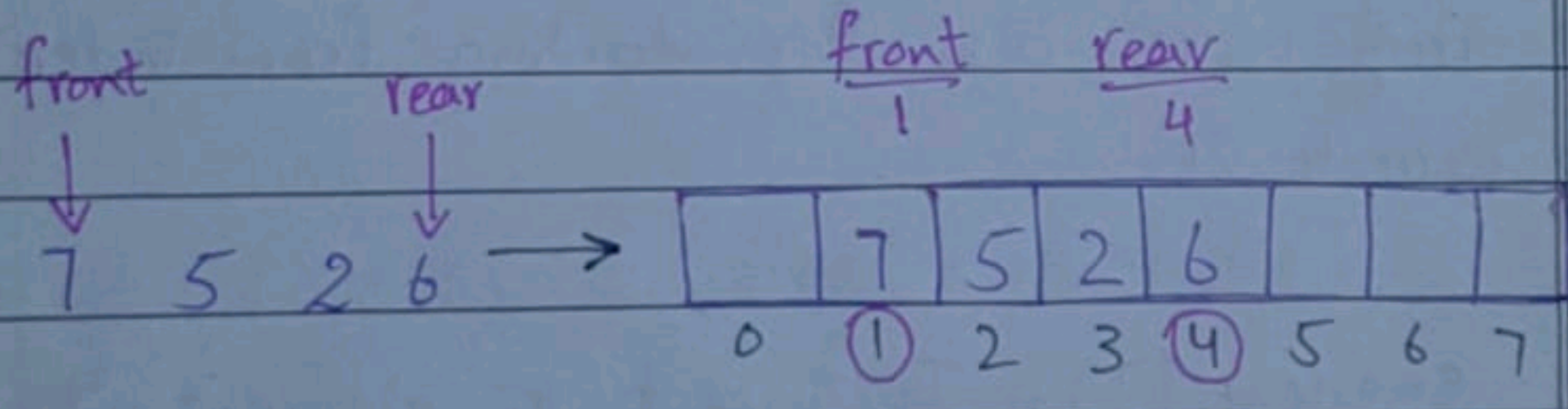
→ enqueue (6): An element i.e 6 inserted in the queue. (value insert at next to rear)



Insertion of element 6

the rear index containing 4 while the front has the same 0 index.

→ dequeue (): When an element is removed from the queue. It is removed from the front index.



Removal of an element from front.

with the removal of element from the queue, we are not shifting the array elements.

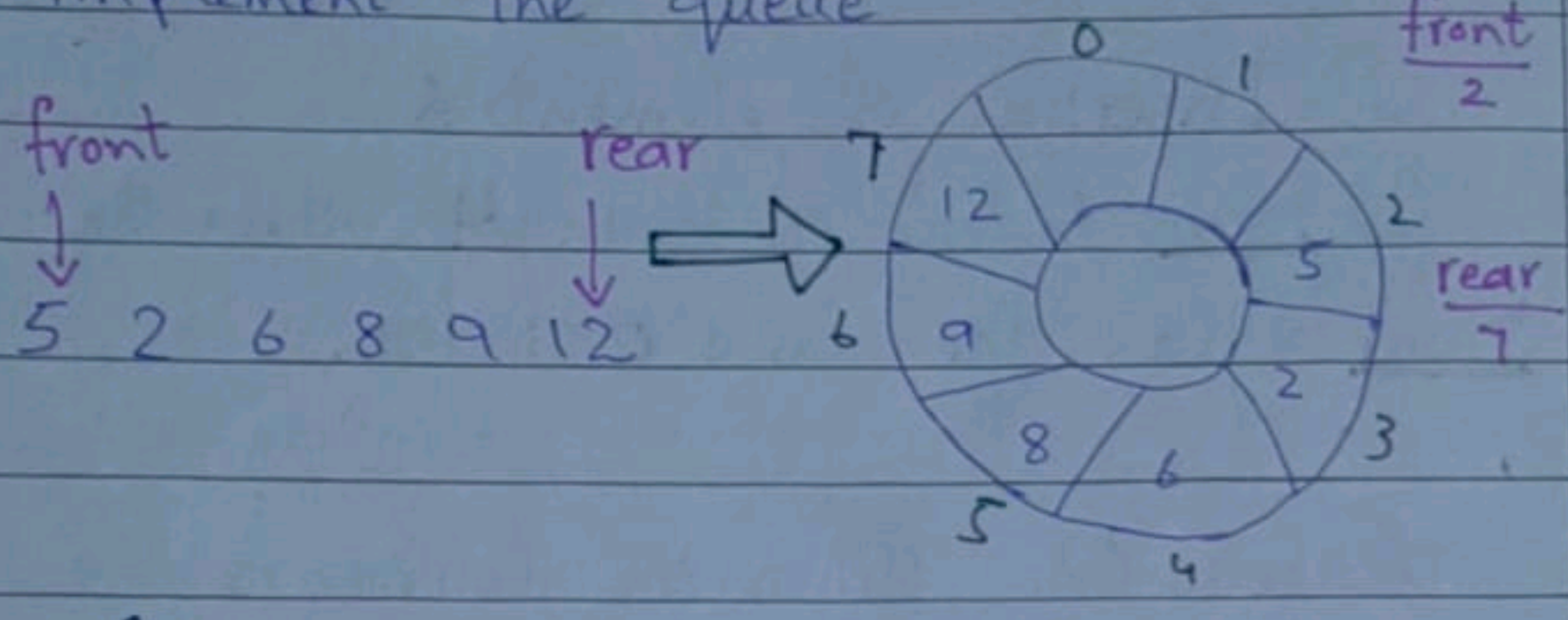
∴ In an array the size is fixed.

& limited size جیسے کچھ Problems دکھتی ہیں جیسے اس میں insert new values کے لیے index 0 کے array آپ کروا سکتے۔ جہاں جگہ خالی پڑی ہوتی ہے

The solution to this Problem lies in allowing the queue to wrap around.

How can we wrap arround?

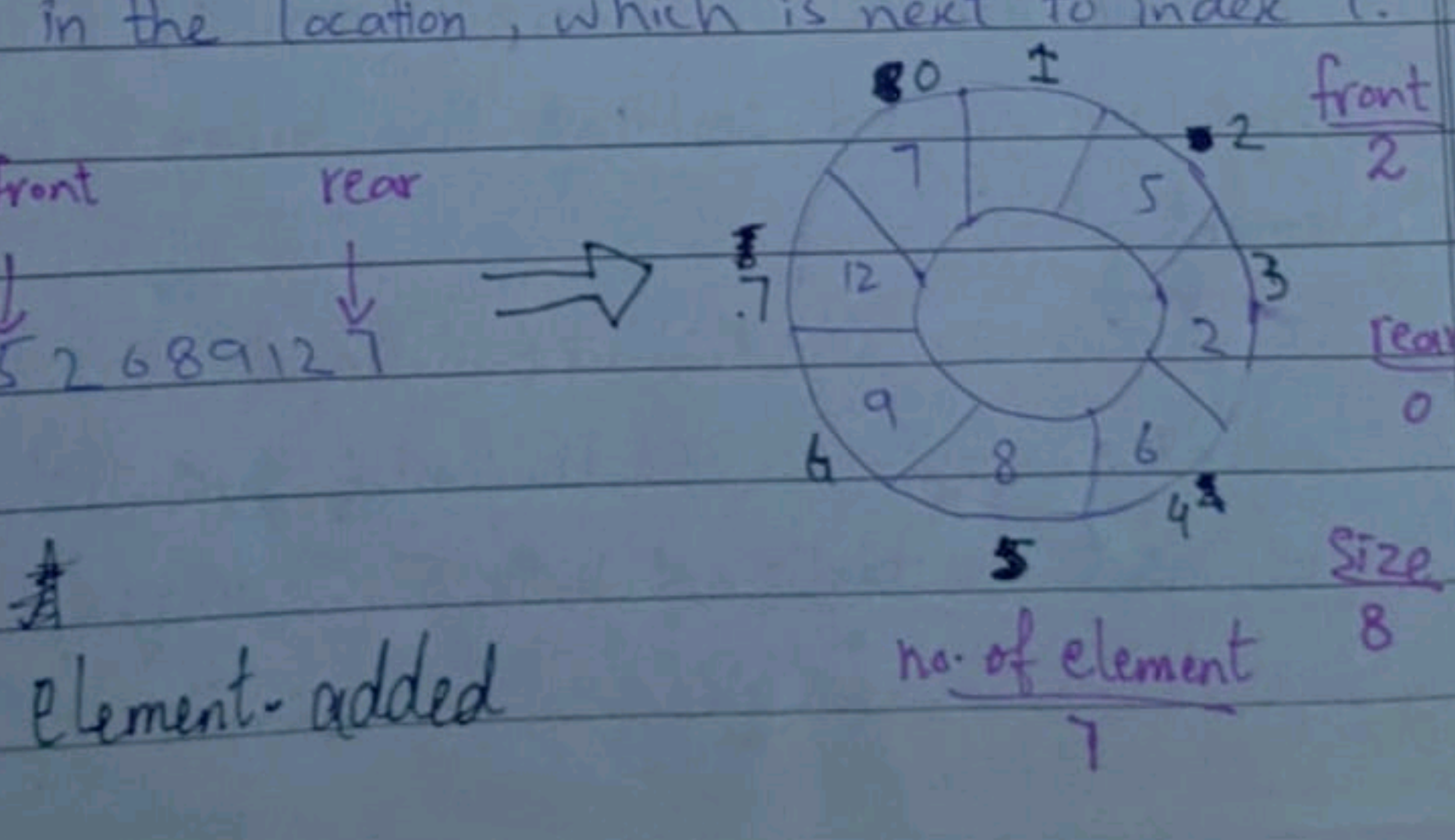
We can use circular array to implement the queue



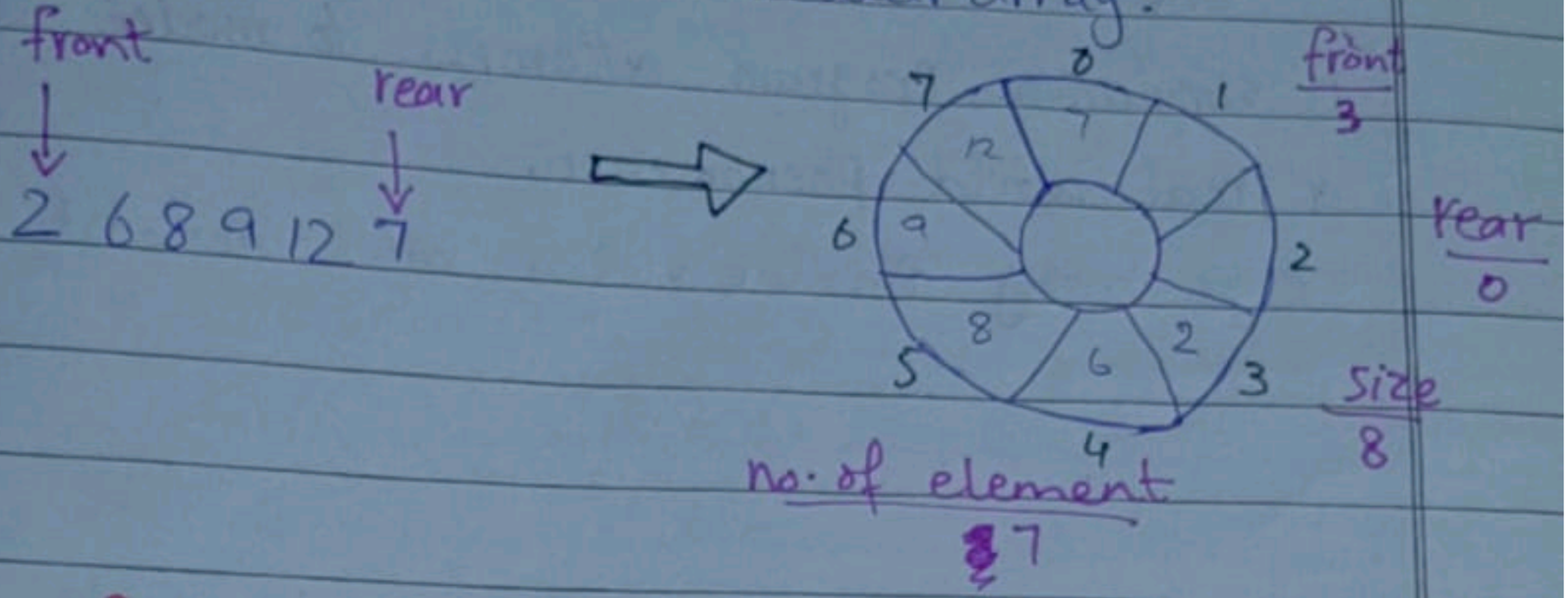
Circular array to implement queue.

The number of locations in the above circular array are also 8, starting from index zero to 7.

enqueue (7): To insert an element 7 in the array, we insert this element in the location, which is next to index 7.



dequeue (): element (from the front) removed from the circular array.



→ isFull(): Return "true" if the no. of element in the array is equal to the size of array.

isFull() function to confirm that there is some space Left in the queue to enqueue() more elements.

→ isEmpty(): Return true, if there is no element in the queue.

★ int isFull()
{ return noElement == size; }

★ int isEmpty()
{ return noElements == 0; }

For Paid Service : +923174545120
CS301 Short Notes
By Qalb-e-Momin

Use of Queues:

One of the most useful is simulation. A simulation program attempts to model a real-world phenomenon.

e.g; many popular videos are simulated.

Simulation Models

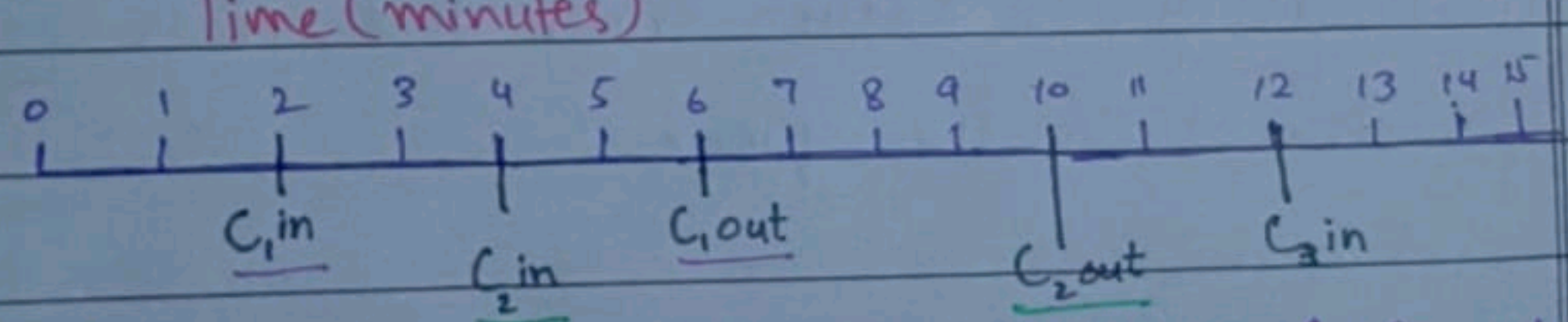
Two common models of simulation are:

1- Time-base simulation

2- Event-based simulation

We have a timeline and marks for every minute.

Time (minutes)



C_1 come at 2 min, C_2 enter at 4min. As C_1 need 4 minutes for his transaction, so he leaves at 6 min. RULE: FIFO (first-in-first out)

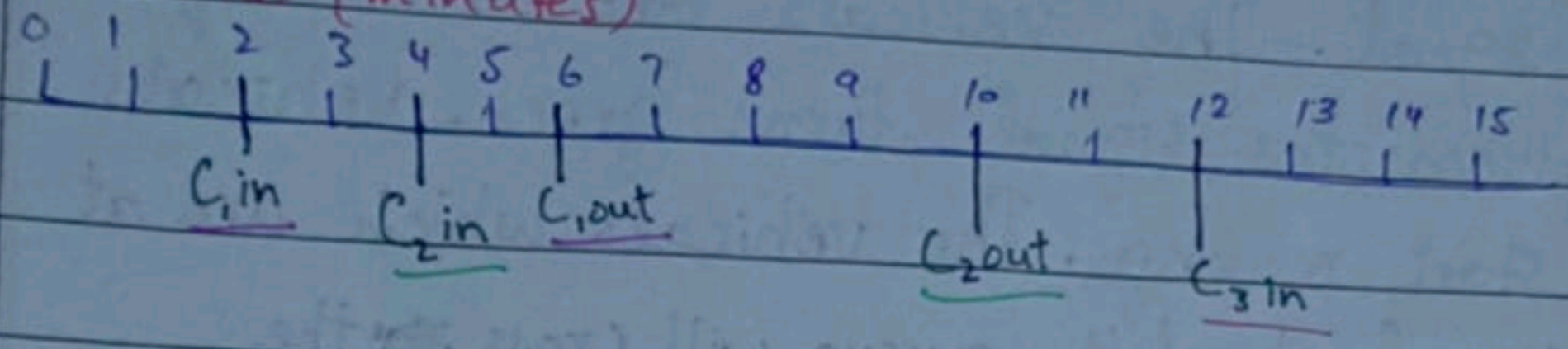
C_2 requires 6 min for the processing so, C_2 leaves at 10 min. This way the activity goes on.

Time-based Simulation

DATE:

Lecture No 10:

Event-based simulation:



Event 1: 2 mins C_1 in

Event 2: 4 mins C_2 in

Event 3: 6 mins C_1 out

Event 4: 10 mins C_2 out

Event 5: 12 mins C_3 in

Here we have a list of events. How can we process them?

We will make a queue of these events. Remove the event with the earlist time from the queue & process it. Insert the newly created events in the queue.

Priority Queue:

As started earlier, the queue is a FIFO (First-in-first-out) structure. But ....

A queue where the dequeue operation depends "not on FIFO", is called a priority queue.

For example:

Traffic is stoped at the signal. The vehicals are in the queue. When the signal turns green, vehicals start moving. The vehicals which are at the front of the queue will cross ~~for~~ the crossing first. Suppose an ambulance comes from behind. Here, ambulance should be given priority. It will by pass the queue and cross the intersection.

M T W T F S

55

DATE:

For Paid Service : +923174545120

CS301 Short Notes

By Qalb-e-Momin

LMS Handling
Lecture Views
Assignments
QUIZZES
GDBs

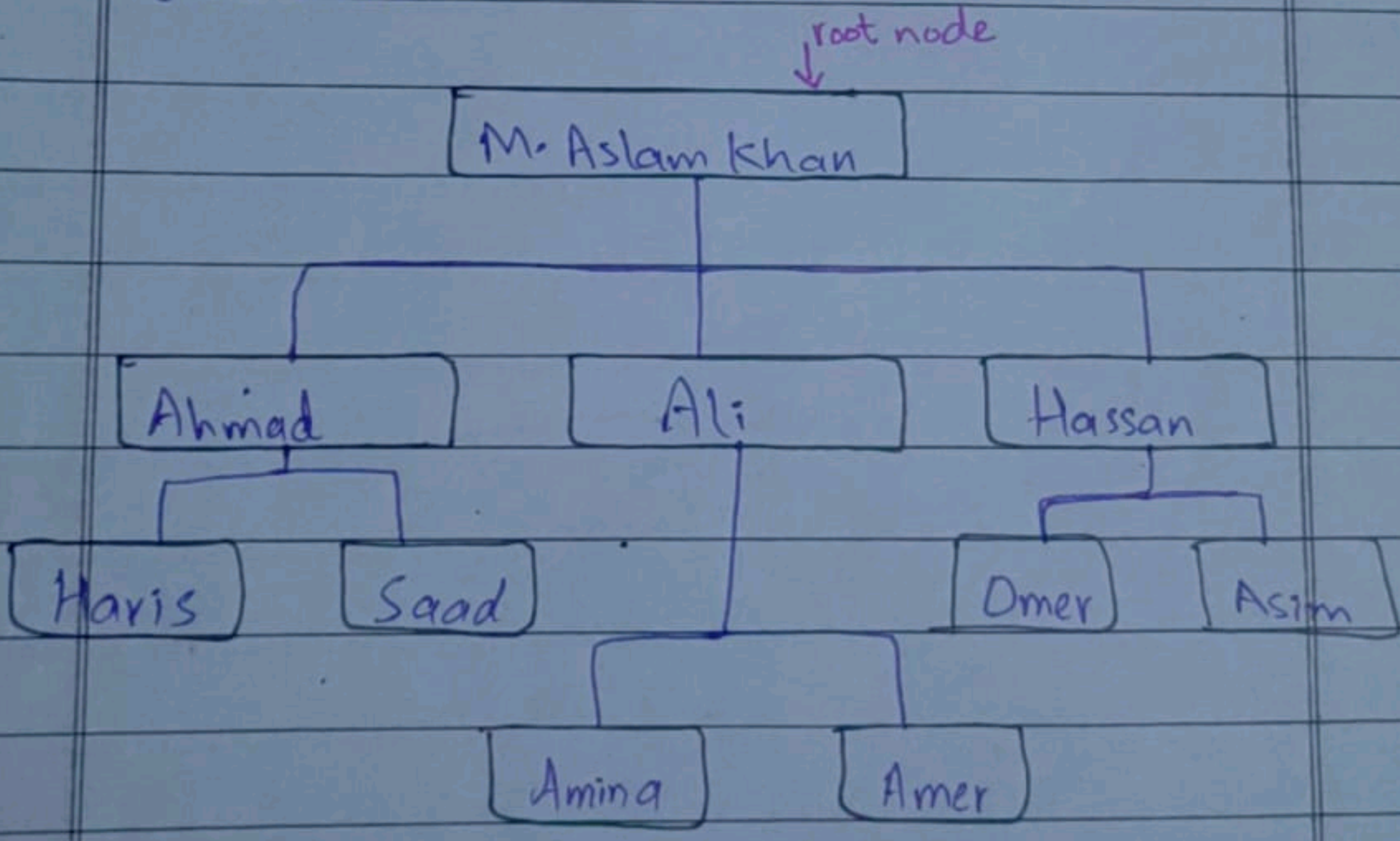
Lecture No 11:

Tree

Tree is one of the non-linear data structures. It is used to represent hirarchical relationships among data.

Structure:

- A tree consist of nodes connected by edges.
- Each tree has a root node at the top, with child nodes branching downward.



Our tree structure is different from the actual tree in a way that the actual tree grows from down-to-up. This top-down structure in CS makes it easy to understand the tree structure

→ Searching in a tree can be significantly faster than in a linear data structure (i.e; stack, linked list, queue).

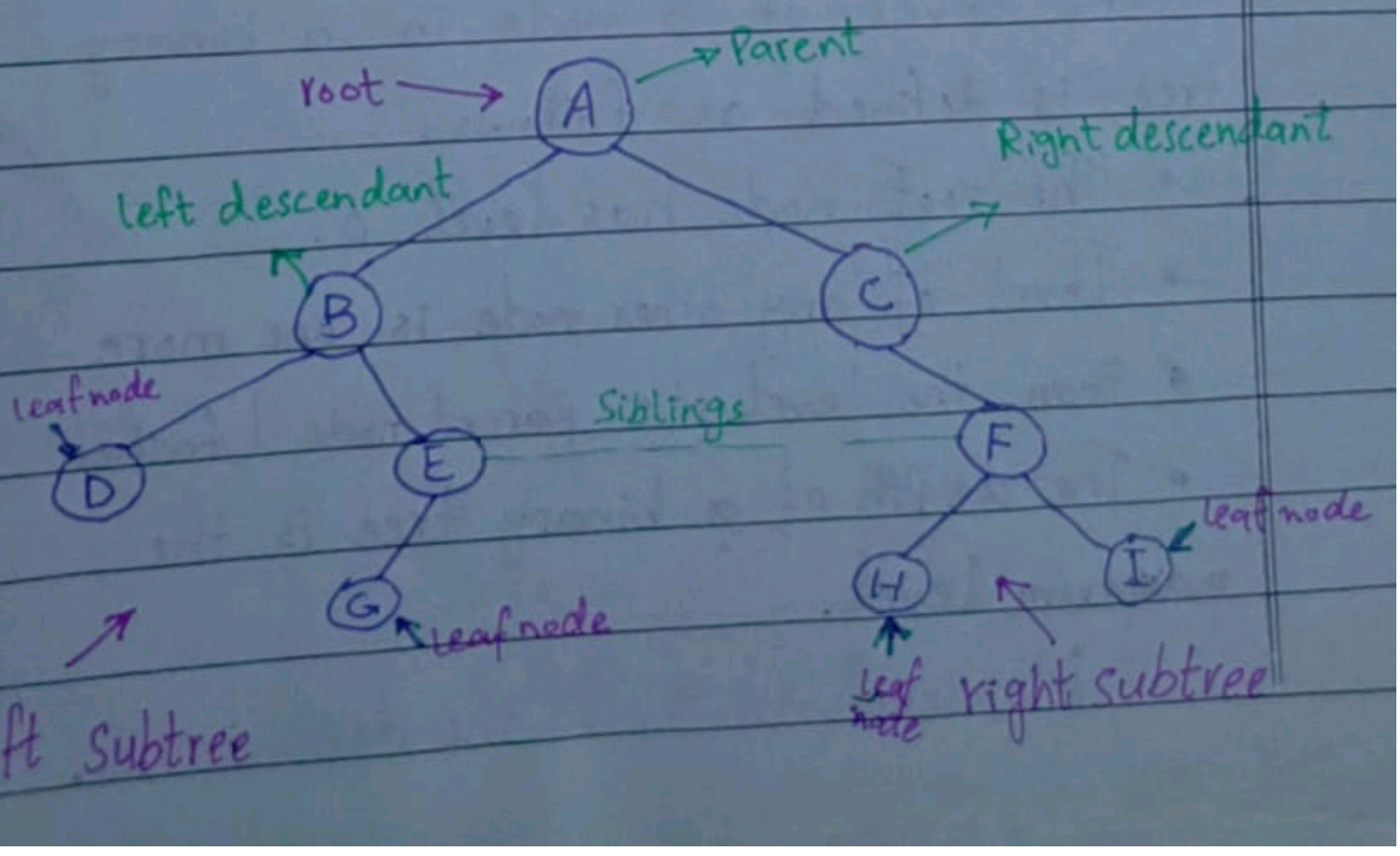
Binary Tree:

A binary tree is a hierarchical data structure in which each node has at most two children, ~~reffe~~ referred to as the left child and the right child.

ایسا tree جس کی parent node کا یا کوئی child نہ ہو، یا ایک ہو، یا زیادہ سے زیادہ 2 child ہوں، binary trees کہتے ہیں۔

* The first subset contain a single element called "root of the tree".
* A node that has no any child called a "leaf node".

Root : top most node

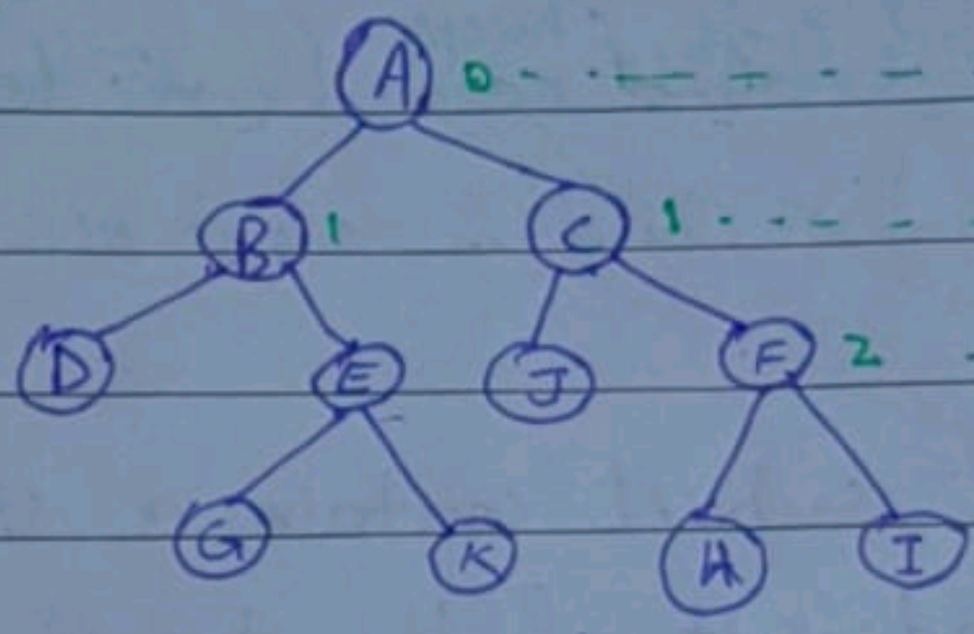


Strictly Binary Tree: (0 or 2) Child

A binary tree is said to be a strictly binary tree if every non-leaf node in a binary tree has non-empty left and right subtree.

(All the non-leaf nodes have their left and right children)

For example:



strictly یہاں ہم نے دو child یعنی تو یہ parent node ہے تبھی اسے binary tree کہا۔

Level

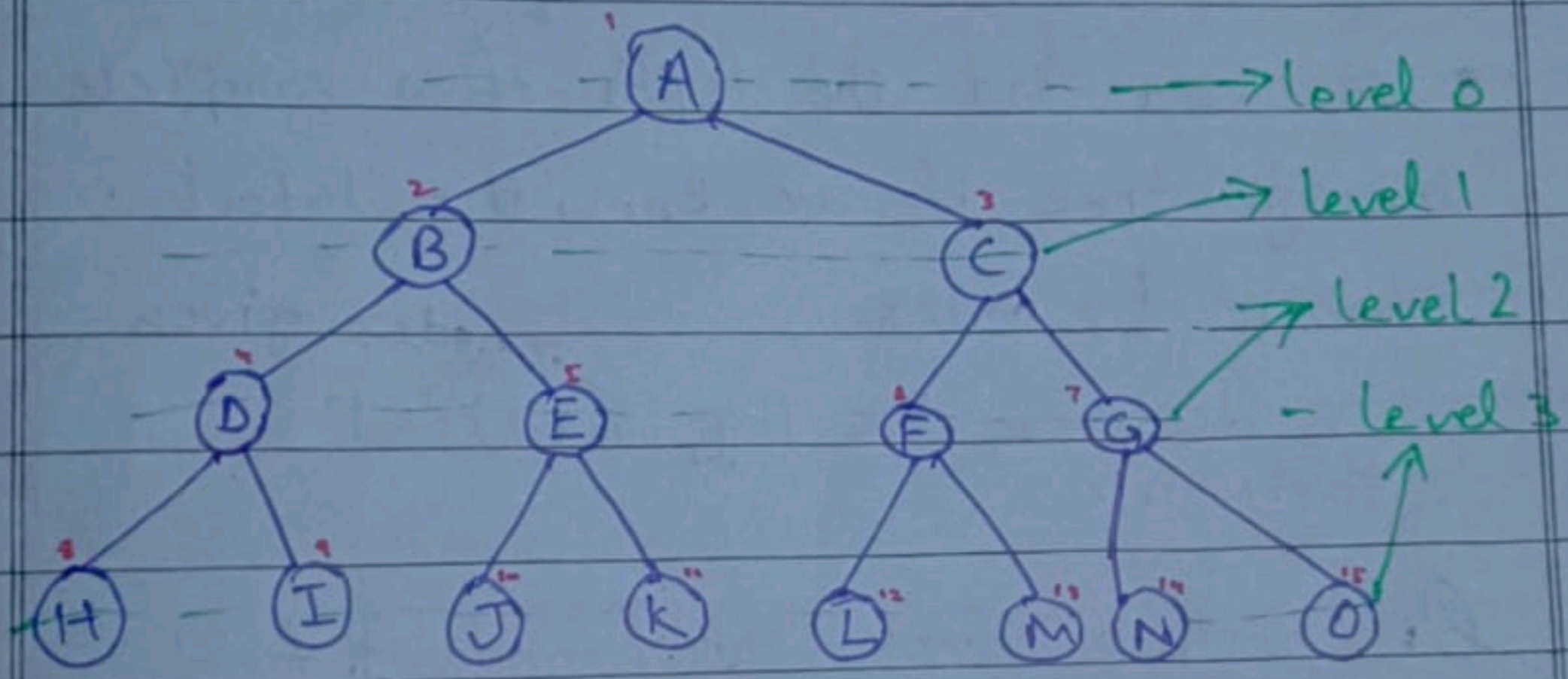
The level of a node in a binary tree is defined as follows:

- The root node has level 0.
- Level of any other node is one more than the level its parent node (father)
- The depth of a binary tree is the maximum level.

Max level is 3, So the depth is 3.

Complete binary tree:

- If there are left and right subtree for each node in the binary tree, it becomes a complete binary tree.
- If all leaf nodes exist at the same level, it becomes a Complete binary tree.
- In a complete binary tree, the nodes increase at each level in a particular order.



Total level = ~~d+1~~ ~~3+1~~ = 4 3

Depth = d = 3 (Max level)

no. of Nodes = N = $2^n = 2^3$ = 8 nodes at level 3.

$2^n = 2^2$ = 4 nodes at level 2.

$2^n = 2^1$ = 2 nodes at level 1.

$2^n = 2^0$ = 1 node at level 0.

Total no. of Node = $2^{d+1} - 1$.

$= 2^{3+1} - 1 \Rightarrow 2^4 - 1 \Rightarrow 16 - 1$

$= 15$

Now, count all the above nodes, which is = 15.

- In a complete binary tree, all the leaf nodes are at the depth level d. (جن کا کوئی child نہیں ہوتا وہ leaf nodes کہلاتے ہیں)

No. of nodes at depth = 2^d

- In a complete binary tree, the difference of total number of nodes ($2^{d+1}-1$) and number of leaf nodes (2^d) will give us the no. of non-leaf nodes.

No. of non-leaf nodes = $2^d - 1$

- We can find the depth of a complete binary tree if we know the total number of nodes.

$$\text{depth of the tree} = d = \log_2(n+1) - 1$$

(n → nodes given)

Q: If you have 100,000 nodes, find the depth of the tree, levels, leaf nodes, non-leaf nodes.

Step-1:

$$d = \log_2(100,000+1) - 1 \approx [16.6096] - 1$$

Depth = 15

Level = $d+1 \Rightarrow 15+1 = 16$ X Level = 15

Leaf Nodes: $2^d \Rightarrow 2^{15} = 32768$

Non-Leaf Nodes: $2^d - 1 \Rightarrow 2^{15} - 1 = 32768 - 1$

$= 32767$

Operations on binary tree.

If p is pointing to a node in an existing tree, then

- lef(p) returns pointer to the left subtree.
- right(p) returns pointer to right subtree.
- parent(p) returns the father of p.
- brother(p) returns the brother of p.
- info(p) returns content of the node.
- setLeft(p,x) & setRight(p,x) Pg#124 from handsout

Tips:

- A priority queue doesn't follow the FIFO rule.

- A complete binary tree is necessarily a strictly binary tree but not vice versa.

- At any level K, there are 2^K nodes

- Total no. of nodes in a complete binary tree of depth d is $2^{d+1} - 1$.

- Leaf nodes = 2^d

- Non-leaf node = $2^d - 1$
  (total no. of nodes − leaf nodes)

Lecture No 12:

Operations on binary tree:

Left (p), right (p), parent (p), brother (p), info(p), setLeft(p, x), setRight (p, x). All these methods are required to build and to retrive values from a tree.

Applications of Binary Tree:

Binary Tree is useful structure when two-way decisions are made at each point.

Searching for Duplicates:

Suppose we want to find all duplicates in a list of the following numbers:

14, 15, 4, 9, 7, 18, 3, 5, 16, 4, 20, 17, 9, 14, 15.

Note: To search a number in a linked List, we have to begin from the start of the list to the end in the linear fashion, traversing each node in it.

Solution:

⟶

It will reduce the number of comparison (B.T).

here the list:

14, 15, 4, 9, 7, 18, 3, 5, 16, 4, 20, 17, 9, 14, 5

For solution, follow some ruler:

1- First number in the list is placed in a root node.

2- We take the "next number", compare it with the root node value.

- If it is the same, this means the presence of a duplicate.
- If it is greater than the root node value, the new node is turned into the right side of the root node.
- If it is less than the root node value, the new node is turned into the left side of the root node.

For Paid Service : +923174545120
CS301 Short Notes
By Qalb-e-Momin

Lecture No 13:

Cost of search

Binary tree maximize the speed of searching.

کسی بھی tree کے اندر search value کرنے کے لیے کتنا ٹائم (time) لگنا پڑتا ہے؟

جتنے آپ کے tree میں no. of levels اتنا time constant لگتا ہے۔

If the tree has 6 levels, the while loop will execute maximum 6 times.

Note: The usage of memory space doesn't cause any effect in the linked list and tree data structure.

(Dynamically memory use in both structures).

Thus, tree structure is better for data storing & it makes the search process faster.

Binary Search Tree:

A binary tree with such a property that items in the left-subtree are "smaller than the root" & items in the right sub-tree are "larger than the root" is called a binary search tree (BST).

- BST is actually a process of storing the data in a sorted form.
- BST is also be termed as ordered tree.

Traversing a binary tree:

→ The way to Print the numbers Present in a BST.

Most commonly traversing methods:

1- PreOrder traversal — Node, Left, Right

2- InOrder traversal — Left, Node, Right

3- PostOrder traversal — Left, Right, Node

* visit node { left subtree, } { * visit Node

{ * visit right subtree, }

- Pre میں پہلے سے ہی Node, left, right
- In میں Node center میں آتا ہے (L,N,R)
- Post میں Node Last میں آتا ہے (L,R,N)

1- **Preorder Traversal (N, L, R)**

In preOrder traversing method, first of all we check whether the pointer provided is NULL or not. If it is not NULL, we print the information stored in that node with the help of getInfo() method. Then we call the getLeft() method that returns a pointer of the left node. After that, the preorder method is called for the right node with getRight().

```
void preorder (TreeNode<int>*treeNode)
{

// 1:   if (treeNode != NULL) {
// 2:      cout << *(treeNode -> getInfo()) << " ";
// 3:      preorder (treeNode -> getLeft());
// 4:      preorder ( treeNode -> getRight());
      }

}
```

Step-1:

سب سے پہلے یہ دیکھیں گے کہ tree میں کوئی node ہے بھی یا نہیں۔

Step-2:

اگر treeNode میں سے کوئی بھی true condition ہے تو print کروا دیں گے۔ getInfo() کی مدد سے rootNode کی value

Step-3: left side سے getLeft() کی مدد سے

Step-4: right side سے getRight() کی مدد سے print value

DATE:

2- InOrder Traversal: (L, N, R)

In this methode, first of all we check Pointer is provided, that's NULL or not. second, if the root node is not NULL then we call getleft() to get the Left node and called the inorder function. Print the left node.

After completely traversing the complete left tree, we print the value of the node. In the end, we traverse the right sub-tree in recursion.

```
void inorder*(TreeNode<int>* treeNode)
{

    if (treeNode != NULL) {
   ~~cout~~ inorder(treeNode -> getleft());
    cout << * (treeNode -> getInfo()) << " ";
    inorder(treeNode -> getRight());
    }
}
```

left sub-tree کی root node پہلے سب سے اس میں سب
پھر print value left سب سے آخر
side right کے پھر اس value کی root node اوپر اس سے
print value ہو گی۔
پھر اسی طرح یہ عمل بار بار root node پر دہرایا
جائے گا (L, N, R)

Postorder Traversal (L, R, N)

In this, the input argument is a pointer to the TreeNode. If the node is not NULL (!=NULL), we traverse the Left sub-tree first.

After that we will traverse the right tree and print the root node (N) value.

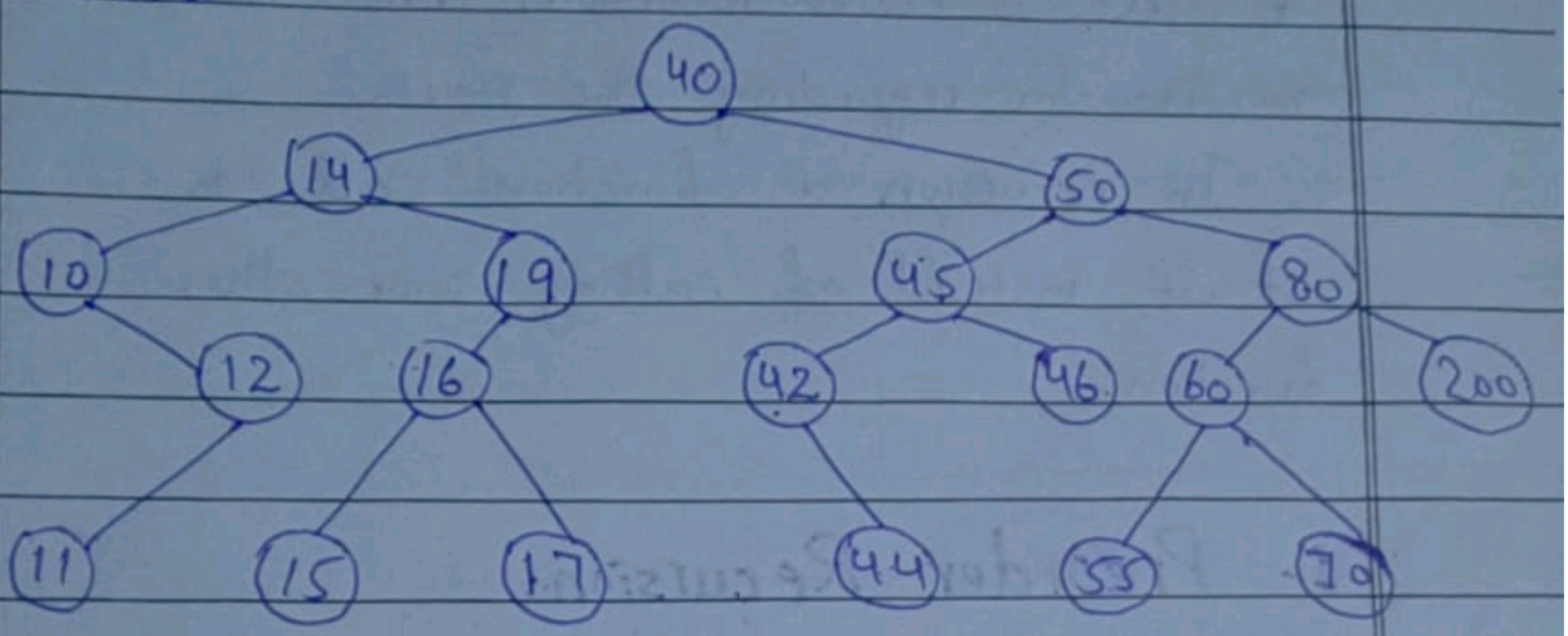
```
void Postorder(TreeNode<int>* treeNode)
{
    if (treeNode != NULL) {
        Post order(treeNode -> getLeft());
        Post order(treeNode -> getRight());
        Cout<<*(treeNode -> get Info())<<" ";
    }
}
```

// As all of these above routines are // functions so we will call them as:

```
Cout << "Preorder:";
preorder (root);
cout << "inorder:";
inorder (root);
cout << " Postorder: ";
Postorder (root);
```

Exercise.

Find out the Preorder, Inorder, Postorder traversal of the tree given below:



Preorder Traversal: (N, L, R)

Inorder Traversal: (L, N, R)

Postorder Traversal: (L, R, N)

Lecture No 14:

Recursive Calls

When a function is calling to itself is called recursion. And there is no problem regarding the stack.

In recursion a function calls to it self instead of calling some other function.

1- Preorder Recursion:

When preorder function calls itself recursively, the nodes are processed in the following order:

1- Visit the root Node

2- Traverse the left sub-tree in preorder.

3- Traverse the right subtree in preorder.

2- Inorder Recursion:

When inorder function calls itself recursively, the nodes are Processed in the following order:

1- Traverse the left subtree first before Printing the root. First left then node, then right tree

2- Afterwards, the right subtree is traversed.

Non-Recursive Traversal:

If we are not using the recursion, how these things can be manged; we have to manage all this In our methods.

We can Implement non-recursive versions of the preorder, inorder and post order traversal by using an explicit stack.

Note: We cannot avoid stack. The stack will be used to store the tree nodes in the appropriate order.

Traversal Trace

Let's compare the recursive inorder and non-recursive inorder:

Recursive inorder: inorder(14);

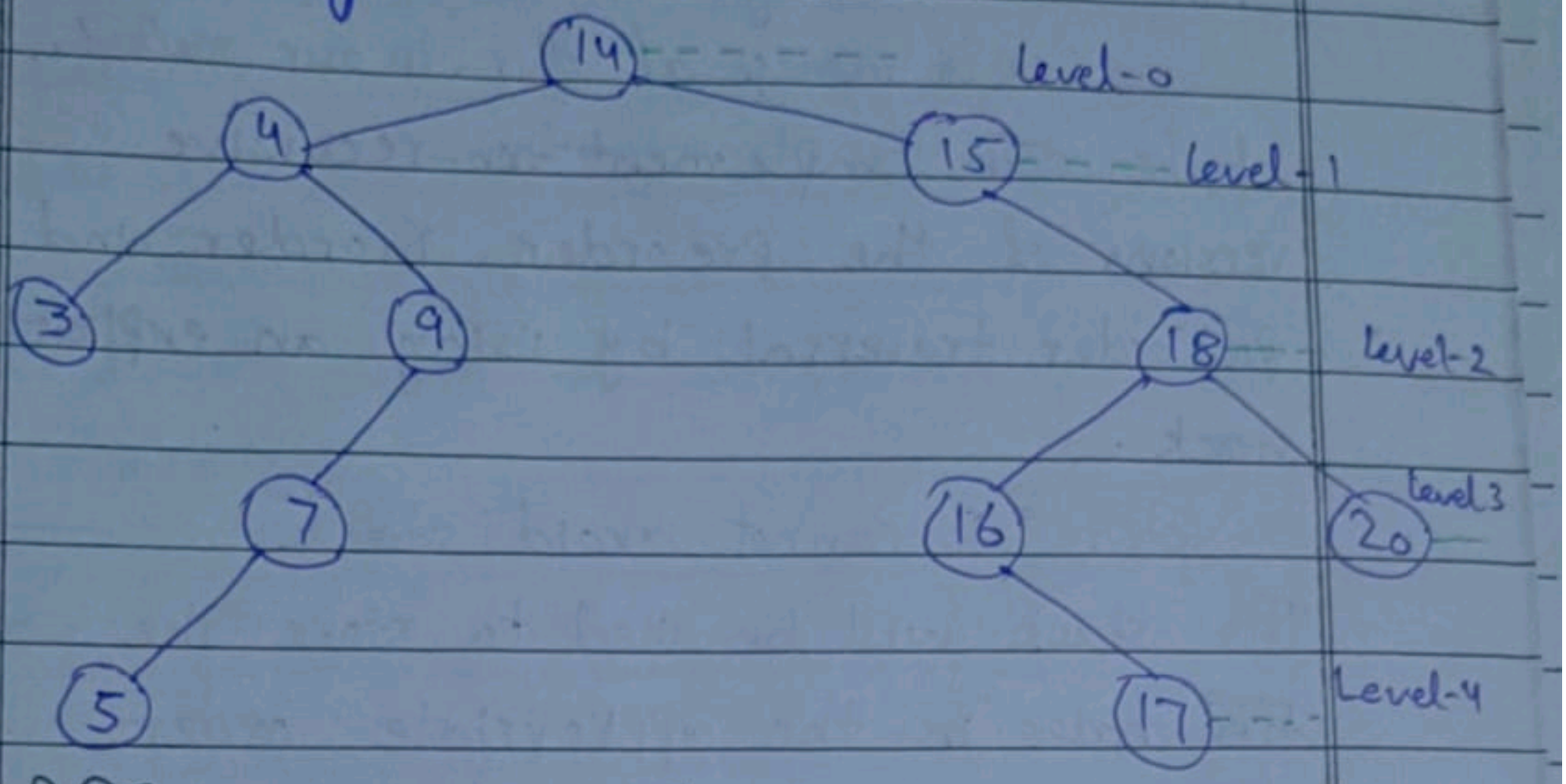
: The function name is inorder() used for insert the value in the tree. We are printing the value according to inorder() order.

Non-Recursive inorder: Push(14);

In case of non-recursive, Push() method is employed.

Lecture: No 15:

Level Order Traversal of a Binary Tree



PRINT a Binary Tree:

14, 4, 15, 3, 9, 18, 7, 16, 20, 5, 17

Note:

A queue is a FIFO structure, can make the level-order traversal easier.

:

Storing Other types of Data in B.T:

We are using int number in binary tree because they were easier to understand for storing and comparison problem. But if we want to enter the name of the people in the telephone directory in the binary tree, we build a binary tree for strings

Binary Search Tree

A binary search tree is a special type of binary tree used for efficient searching, insertion and deletion operations. It maintains a sorted structure

A Binary Search Tree is a binary tree where each node has:

- A key (value)
- Left child with values less than the node.
- Right child with values greater than the node.

Binary Search Tree with Strings

Building a BST with strings allows automatic sorting via "inorder traversal". This methods show how tree structure can be used for sorting and organizing data.

Purpose:

✔ To insert and store words (strings) in a binary search tree.

✔ To print the words in sorted order using inorder traversal.

Although functional, BST sorting is not always the most efficient (compared to other sorting algorithms).

Deleting a Node From BST:

Suppose you want to delete one of nodes from BST.

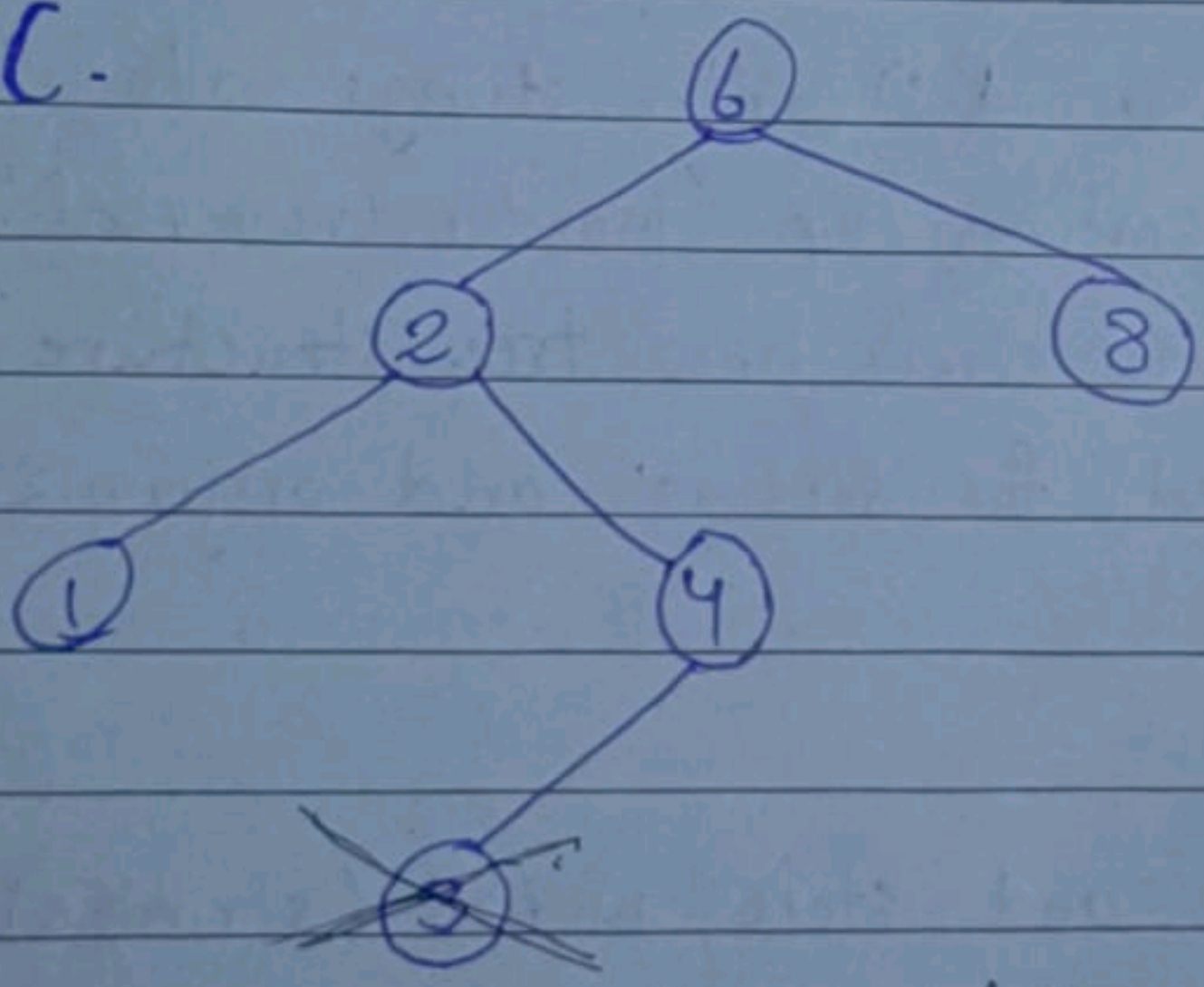
There are 3 cases for deleting a node from BST.

Case I:

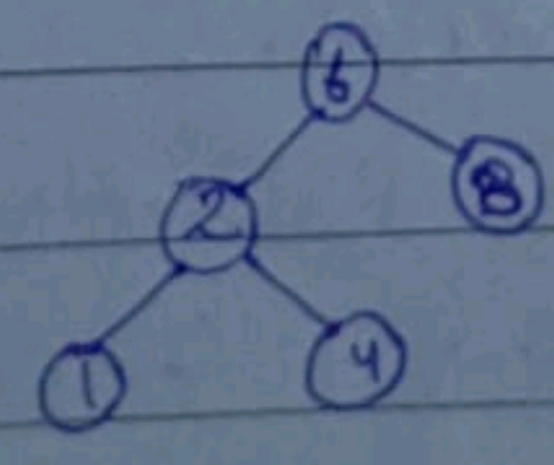
The node to be deleted is the leaf node (has no child).

Simply delete it. (if the memory for the node has been dynamically allocated, we will release it.

C.



Suppose, we want to delete the leaf node containing number 3. we delete the leaf node and point the right subtree pointer to NULL.

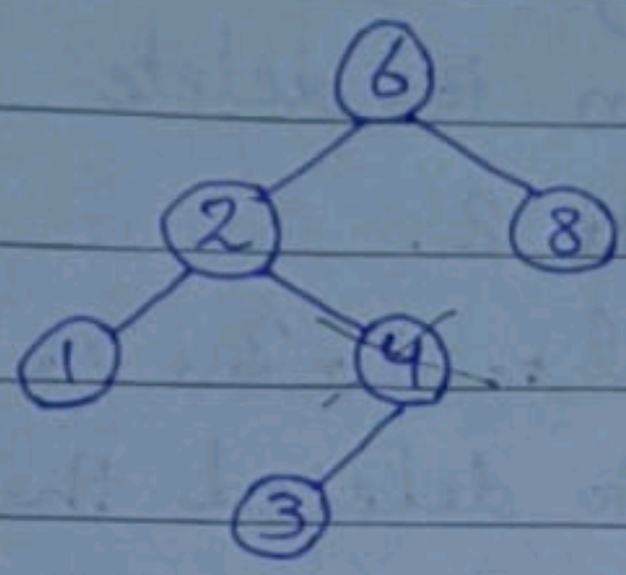


Leaf node deleted.

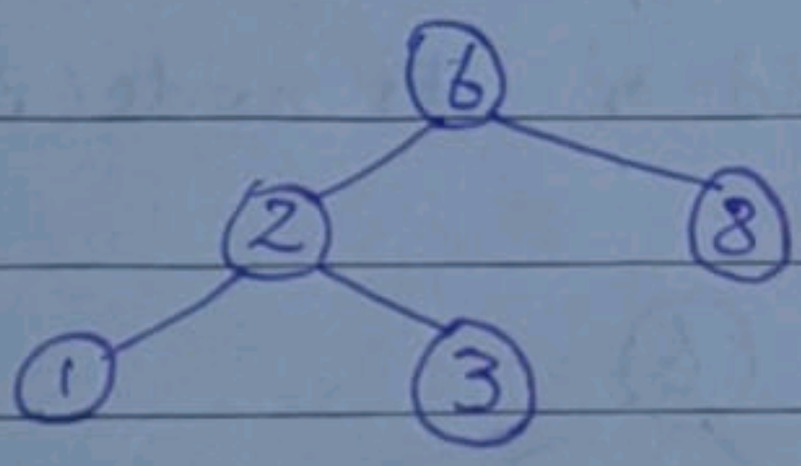
Case II:

If the node to be deleted has one child either left child (subtree) or right child (subtree).

Replace node with its only child.



Suppose we want to delete the node containing number 4. Then, we have to adjust the child of that's node by replacing it.

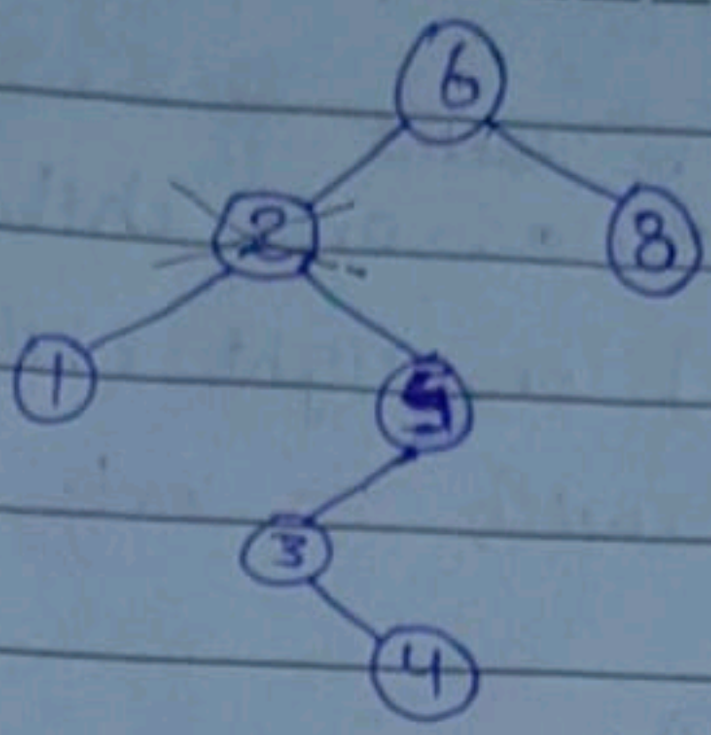


Case III:

The case 3 is bit complicated, when the node to be deleted has both the left and right children (subtree).

Most difficult case.

Replace node's value with it's inorder successor (minimum in right subtree), then delete the successor.



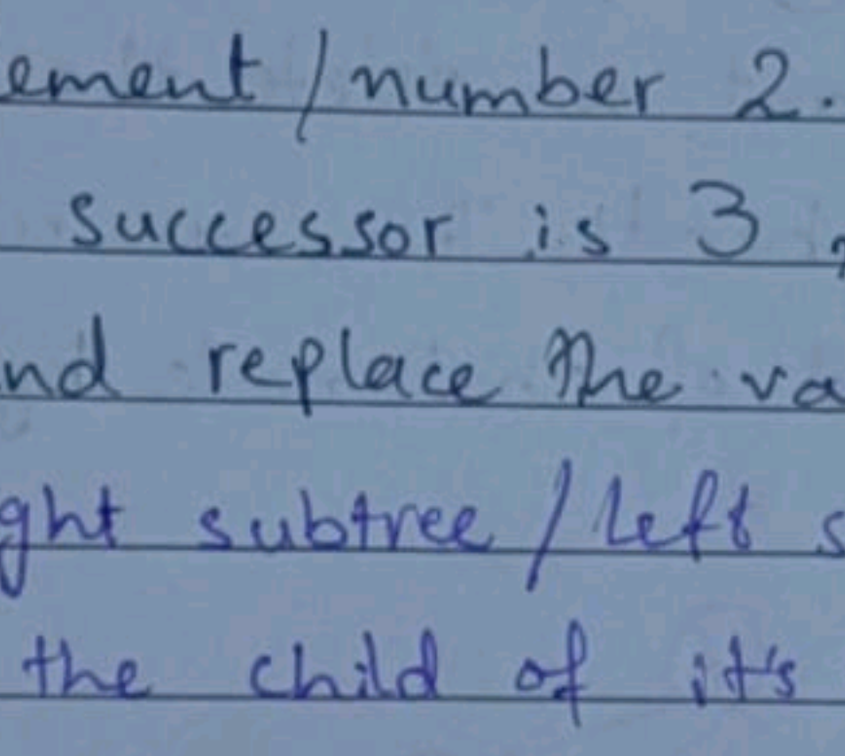
Suppose we wan to delete the node containing number 2.

L,N,R Inorder traversal :- 1, 2, 3, 4, 5, 6, 8

Now, we have to deleted the node with element / number 2.

Inorder successor is 3, delete this node and replace the value.

And right subtree / left subtree of this node is the child of it's node (parent).



L,N,R Now, inorder traversal is: 1, 3, 4, 5, 6, 8.

M T W T F S

DATE:_____

Lecture no.16,7,18 missed....

صَلَّى اللهُ عَلٰى حَبِيْبِهٖ سَيِّدِنَا مُحَمَّدٍ

وَّاٰلِهٖ وَاَصْحَابِهٖ وَسَلَّمْ

Exam ki Kaamyaabi ki Dua

فَاِنَّ حَسْبَكَ اللّٰهُ ؕ

هُوَ الَّذِیْۤ اَیَّدَكَ بِنَصْرِهٖ وَبِالْمُؤْمِنِیْنَ

سورہ انفال

Dua : Fa inna hasbakallahu huwal lazi ayyadaka binasrihi wa bil mu'mineen

(Al Quran 8.62)

Tareeka : Imtihaan se pehle Awwal Aakhir me durood sarif padh kar 7 Martaba ye Dua padhe..